

RÉPUBLIQUE DU SÉNÉGAL
UNIVERSITÉ GASTON BERGER DE SAINT-LOUIS
U.F.R DES SCIENCES APPLIQUÉES ET TECHNOLOGIES

Département Informatique



Mémoire présenté pour l'obtention du diplôme de Master en Informatique
Option : Gestion de Données et Ingénierie logicielle (GDIL)

Conception d'une plateforme numérique personnalisée
pour la gestion des projets et activités scientifiques du Centre de
Recherche Agricole de Saint-Louis ISRA/CRA-SL

Présenté par

Mouhamadou Lamine Bathily

Lieu de stage :

ISRA/CRA Saint-Louis



Encadré et supervisé par :

Dr Mamadou Ndiaye (ISRA/CRA Saint-Louis)

Pr Cheikh Mouhamadou Bamba Dione (UGB)

Soutenu le 21 Janvier 2025 devant le jury composé de :

Nom jury	Président	Professeur titulaire
Nom2 NOM2	Rapporteur	Maître de conférences
Nom3 NOM3	Examineur	Maître-assistant
Nom4 NOM4	Membre	Docteur

Année Universitaire : 2023-2024

Dédicace

*À la mémoire de mon père, **Abdoulaye Bathily**,
décédé depuis mon enfance,
dont l'absence a forgé ma détermination
et dont les valeurs continuent d'inspirer mon parcours ;*

*à mon oncle, **El Hady Mor Mbengue**,
qui m'a éduqué, soutenu et accompagné
avec engagement et bienveillance tout au long de mon parcours académique ;*

*et à l'ensemble de ma famille,
pour leur amour, leurs encouragements et leur soutien indéfectible.*

Remerciements

Ce mémoire de Master est l'aboutissement d'un parcours académique et humain rendu possible grâce au soutien et à l'accompagnement de nombreuses personnes, à qui j'adresse ma profonde gratitude.

Je remercie tout d'abord le **Centre de Recherches Agricoles (CRA) de Saint-Louis** de l'Institut Sénégalais de Recherches Agricoles (ISRA) pour son accueil et le cadre de travail favorable offert. J'exprime mes sincères remerciements à son Directeur, **Dr Omar Ndaw Faye**, pour sa disponibilité et l'intérêt porté à ce travail.

Je tiens à remercier particulièrement mon maître de stage, **Dr Mamadou Ndiaye**, pour son encadrement, ses conseils et la confiance qu'il m'a accordée tout au long de ce projet. Mes remerciements s'adressent également à mon encadreur académique, **Dr Mouhamadou Bamba Dione**, pour son suivi rigoureux et ses orientations méthodologiques.

Je remercie l'ensemble des chercheurs, cadres et techniciens du CRA de Saint-Louis pour leur collaboration et leurs échanges enrichissants, avec une mention spéciale à **Mbathio Ba Diallo** pour son soutien et son esprit de coopération. J'exprime également ma reconnaissance à **Dr Khadim Dramé**, ancien professeur, pour la qualité de son enseignement et son influence positive sur mon parcours.

Je remercie mes amis **Hamadou Ba**, **Baboucar Badji**, **Ndary Ka** et **Abdou Salam Diallo** pour leur soutien, leur amitié et leurs encouragements.

Enfin, ma gratitude va à ma famille, dont le soutien a été une source constante de motivation. Je remercie tout particulièrement ma mère, **Ndella Mbengue**, pour son amour et ses sacrifices. Une pensée spéciale est adressée à mes tantes **Fatou Mbengue** et **Mbenda Diop** pour leur rôle essentiel dans mon éducation et ma réussite. Je remercie également mes sœurs **Oulimata Bathily** et **Aida Sarr**, ainsi que mes cousins **Adama Diop**, **Ousmane Diop**, **Adja Marie Mbengue** et **Malick Sarr** pour leur soutien.

À toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail, j'exprime ma reconnaissance la plus sincère.

Résumé

Le Centre de Recherches Agricoles (CRA) de Saint-Louis, structure de l’Institut Sénégalais de Recherches Agricoles (ISRA), produit chaque année une quantité importante de données scientifiques, de rapports techniques et de documents issus des activités de recherche. Toutefois, ces informations sont souvent dispersées, peu structurées et difficilement accessibles, ce qui limite leur exploitation, leur partage et leur pérennisation.

Dans un contexte marqué par la digitalisation croissante des processus scientifiques, ce mémoire a pour objectif de concevoir une plateforme web collaborative dédiée à la gestion des projets de recherche, des activités scientifiques, des données et des documents au sein du CRA de Saint-Louis. La solution proposée vise à offrir à chaque profil d’utilisateur (chercheur, coordonnateur, technicien, assistant de recherche et administrateur) un espace de travail personnalisé et adapté à ses responsabilités.

La démarche adoptée repose sur une analyse approfondie des besoins, la définition de spécifications fonctionnelles et non fonctionnelles, ainsi que sur une modélisation UML du système. Plusieurs diagrammes UML, notamment les diagrammes de cas d’utilisation, de classes, de séquences, de composants et de déploiement, ont permis de formaliser la structure et le comportement du système.

La plateforme développée s’appuie sur une architecture 3-tiers, assurant une séparation claire entre la couche de présentation, la couche métier et la couche d’accès aux données. Le frontend est réalisé avec React, Vite et TypeScript, tandis que le backend repose sur Node.js, Express.js et TypeScript, avec l’ORM Prisma et la base de données PostgreSQL pour la persistance des données. Le modèle de programmation asynchrone et non bloquant adopté garantit de bonnes performances, une forte réactivité et une scalabilité adaptée aux besoins du CRA.

Les résultats obtenus montrent que la plateforme contribue à une meilleure centralisation des informations scientifiques, facilite la collaboration entre les acteurs de la recherche et améliore le suivi et la valorisation des activités scientifiques. Ce travail participe ainsi à la modernisation du système d’information du CRA et propose une solution numérique évolutive et durable au service de la recherche agricole.

Mots-clés : Plateforme web, Système d’information, Recherche agricole, Architecture 3-tiers, Node.js, React, UML.

Abstract

The Centre de Recherches Agricoles (CRA) of Saint-Louis, which is part of the Senegalese Institute of Agricultural Research (ISRA), produces a large volume of scientific data, technical documents and research reports each year. However, these resources are often dispersed, poorly structured and difficult to access, which limits their exploitation, sharing and long-term preservation.

In a context marked by the growing importance of digitalization in scientific research, this work aims to design collaborative web platform dedicated to the management of research activities, scientific data and documents within the CRA of Saint-Louis. The proposed solution provides each user profile (researcher, coordinator, technician, assistant and administrator) with a personalized workspace adapted to their roles and responsibilities.

This master's thesis follows a rigorous methodological approach combining requirements analysis, functional and non-functional specifications, and UML modeling. Several UML diagrams, including use case, class, sequence, component and deployment diagrams, were used to formally represent the system's structure and behavior.

The proposed solution is based on a three-tier architecture, separating the presentation layer, the business logic layer and the data access layer. The frontend is developed using React, Vite and TypeScript, while the backend relies on Node.js, Express.js and TypeScript, with Prisma ORM and PostgreSQL for data persistence. An asynchronous and non-blocking communication model is adopted to ensure performance, scalability and responsiveness.

The results obtained demonstrate that the developed platform improves the centralization of scientific information, enhances collaboration between researchers and facilitates the monitoring and evaluation of research activities. This work contributes to the modernization of information management at the CRA and provides a scalable and sustainable digital solution for agricultural research institutions.

Keywords : Web platform, Information system, Agricultural research, Three-tier architecture, Node.js, React, UML.

Notations et Abréviations

- **API** : Application Programming Interface
- **CRA** : Centre de Recherches Agricoles
- **CSRF** : Cross-Site Request Forgery – Attaque consistant à forcer un utilisateur authentifié à exécuter une action non intentionnelle sur une application Web.
- **HTTP** : HyperText Transfer Protocol
- **IDE** : Integrated Development Environment
- **ISRA** : Institut Sénégalais de Recherches Agricoles
- **JWT** : JSON Web Token
- **MVC** : Model–View–Controller
- **ORM** : Object-Relational Mapping
- **PWA** : Progressive Web Application
- **REST** : Representational State Transfer
- **UML** : Unified Modeling Language
- **XSS** : Cross-Site Scripting – Faille de sécurité permettant l’injection de code malveillant dans les pages Web affichées aux utilisateurs.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Notations et Abréviations	v
Introduction	1
1 Présentation Générale	2
1.1 Présentation de la structure d'accueil	2
1.2 Présentation du sujet	2
1.3 Démarche Méthodologique	3
2 État de l'art	5
2.1 Systèmes d'information pour la recherche agricole	5
2.2 Plateformes collaboratives et gestion de projets scientifiques	8
2.3 Technologies web modernes pour les systèmes d'information	10
2.4 Sécurité et gestion des accès dans les systèmes scientifiques	18
2.5 Positionnement de la solution proposée	20
3 Spécifications et Analyse des besoins	24
3.1 Spécifications	24
3.2 Analyse des besoins	27
3.3 Diagrammes UML	28
4 Conception de la solution	45
4.1 Architecture logique	45
4.2 Architecture technique	49
5 Mise en oeuvre et résultats	52
5.1 Environnement de développement	52
5.2 Présentation et analyse des résultats	55
Conclusion et Perspectives	66
Références	69

Table des figures

3.1	Diagramme des cas d'utilisation — Vue Globale Modulaire	29
3.2	Diagramme des cas d'utilisation — Module Authentification et Gestion Utilisateurs . .	29
3.3	Diagramme des cas d'utilisation — Module Gestion des projets	30
3.4	Diagramme des cas d'utilisation — Module Gestion des activités	30
3.5	Diagramme des cas d'utilisation — Module Publication	31
3.6	Diagramme de classes — Modèle Utilisateurs et Authentification	32
3.7	Diagramme de classes — Modèle Projets et Activités	33
3.8	Diagramme de classes — Modèle Convention et Partenariat	34
3.9	Diagramme de classes — Modèle Documents et Données	35
3.10	Diagramme de classes — Modèle Collaboration et Séminaires	36
3.11	Diagramme de classes — Module Administration et audit	37
3.12	Diagramme de séquence — Processus de Connexion	38
3.13	Diagramme de séquence — Processus de Connexion WebSocket	39
3.14	Diagramme de séquence — Processus de création de projet	40
3.15	Diagramme de séquence — Collete de données	41
3.16	Diagramme de composants — Architecture Logicielle de la Plateforme	42
3.17	Diagramme de déploiement — Architecture Physique du Système	43
4.1	Architecture de la couche de présentation de l'application CRA	46
4.2	Architecture de la couche métier et données – API Node.js / Express/Prisma/PostgreSQL	48
4.3	Architecture technique 3-tiers de la solution (Frontend React – Backend Node.js/Express – Données PostgreSQL)	50
5.1	Environnement de développement Visual Studio Code utilisé pour le projet	53
5.2	Dépôt GitHub du projet et historique des versions	53
5.3	Conteneurisation de l'application à l'aide de Docker	54
5.4	Documentation interactive des API REST via Swagger	55
5.5	Page d'accueil de la plateforme CRA	56
5.6	Interface d'authentification de la plateforme	57

5.7	Tableau de bord principal de l'utilisateur	57
5.8	Interface de gestion des projets de recherche	58
5.9	Gestion des activités de recherche	59
5.10	Interface de génération de rapports pour le coordonnateur de projet	60
5.11	Gestion des documents scientifiques pour les chercheurs	60
5.12	Module de publications et de transfert de connaissances	61
5.13	Gestion des formations	62
5.14	Système de calendrier et de planification	62
5.15	Gestion des utilisateurs par l'administrateur	63
5.16	Gestion du planning global	64
5.17	Module d'audit des actions utilisateurs	64

Liste des tableaux

2.1	Tableau comparatif des ORM Node.js	15
2.2	Synthèse comparative des approches existantes	21

Introduction

L'agriculture occupe une place centrale dans le développement économique et social du Sénégal. Elle constitue non seulement une source essentielle de revenus pour les populations rurales, mais aussi un levier stratégique pour atteindre la sécurité alimentaire et la souveraineté nationale. Cependant, les transformations rapides observées dans ce secteur exigent aujourd'hui une adaptation constante des pratiques, des méthodes scientifiques et des outils de gestion utilisés par les institutions de recherche.

Le Centre de Recherches Agricoles (CRA) de Saint-Louis, structure de l'Institut Sénégalais de Recherches Agricoles (ISRA), joue un rôle majeur dans la production et la diffusion des connaissances scientifiques destinées à améliorer la productivité et la durabilité des systèmes de production dans la vallée du fleuve Sénégal. Malgré son importance, le CRA fait face à plusieurs contraintes dans la gestion de ses données et activités scientifiques. Les rapports, fiches d'activités et données expérimentales sont souvent dispersés, conservés localement, et difficilement accessibles aux équipes de recherche. Cette situation limite la capitalisation des résultats, la collaboration entre chercheurs et la valorisation des productions scientifiques.

Dans un contexte marqué par la transition numérique et la recherche d'efficacité organisationnelle, l'utilisation des technologies de l'information s'impose désormais comme un levier incontournable. La mise en place d'une plateforme numérique collaborative apparaît ainsi comme une solution pertinente pour moderniser la gestion des activités scientifiques, faciliter la centralisation des données et renforcer la communication au sein du centre.

C'est dans cette optique que s'inscrit le présent mémoire, réalisé dans le cadre de mon stage de fin d'études au CRA de Saint-Louis. Ce travail vise à concevoir une application web personnalisée dédiée à la gestion des activités scientifiques, des projets, des documents et des données des chercheurs. L'objectif est d'apporter une solution simple, efficace et adaptée au contexte local, tout en contribuant à la valorisation des résultats de la recherche et à la visibilité du centre.

L'ensemble de cette étude s'articule autour d'une question centrale : comment concevoir une solution numérique durable et adaptée permettant aux acteurs de la recherche agricole de mieux organiser, valoriser et partager leurs productions scientifiques au sein du CRA de Saint-Louis ?

Présentation Générale

1.1 Présentation de la structure d'accueil

Le Centre de Recherches Agricoles (CRA) de Saint-Louis est une structure de l'Institut Sénégalais de Recherches Agricoles (ISRA). Il a pour mission principale de contribuer à l'amélioration de la productivité agricole, à la sécurité alimentaire et à la résilience des systèmes de production dans la vallée du fleuve Sénégal.

Le CRA joue un rôle central dans la recherche appliquée et le développement de solutions innovantes adaptées aux conditions agroécologiques locales. Ses travaux s'articulent autour de plusieurs thématiques majeures telles que la sélection variétale (notamment le riz et le blé), la gestion durable des ressources naturelles (sols et eau), la protection des cultures, la transformation agroalimentaire et l'agroforesterie.

Le centre mène également des programmes structurants sur la production de semences, l'intensification rizicole, la culture du blé et l'agroécologie. Par ailleurs, il collabore avec de nombreux partenaires nationaux et internationaux afin de renforcer la production scientifique et technique au service de l'agriculture sénégalaise.

Ses principales missions consistent à :

- Générer des connaissances scientifiques et techniques adaptées aux écosystèmes de la vallée ;
- Développer des innovations agricoles favorisant une production durable et compétitive ;
- Encadrer les producteurs dans l'adoption des résultats de la recherche ;
- Participer à la formation des étudiants, chercheurs et agents de développement ;
- Promouvoir une gestion durable des ressources naturelles et des systèmes de production.

1.2 Présentation du sujet

1.2.1 Contexte

Le CRA de Saint-Louis produit chaque année une grande quantité de données scientifiques, de rapports, de fiches d'activités et de documents techniques issus des travaux de recherche menés par les

chercheurs et techniciens. Cependant, ces informations sont souvent dispersées, conservées localement et difficilement accessibles. Cette situation limite la capitalisation, le partage et la valorisation des résultats scientifiques, surtout lors du départ de certains agents ou du renouvellement du personnel.

Dans un contexte où la digitalisation des processus scientifiques devient un levier essentiel d'efficacité, il est nécessaire de concevoir une plateforme numérique collaborative, moderne et sécurisée. Cette solution doit permettre à chaque agent (chercheur, cadre, technicien supérieur, ingénieur ou assistant de recherche) de disposer d'un espace personnel de travail pour organiser, suivre et partager ses activités professionnelles.

1.2.2 Problématique

Malgré l'importance stratégique du CRA dans le dispositif national de recherche agricole, il ne dispose pas encore d'un système d'information intégré permettant la gestion centralisée des activités scientifiques et techniques. Les difficultés rencontrées se traduisent notamment par :

- Une dispersion et une faible accessibilité des données et documents de recherche ;
- L'absence d'un système collaboratif favorisant le travail en équipe et le partage d'informations ;
- Une gestion manuelle des projets et des activités rendant le suivi difficile ;
- Un manque d'outils d'analyse et de visualisation des indicateurs de recherche.

Ces constats soulignent la nécessité de mettre en place une plateforme numérique adaptée aux besoins spécifiques du CRA, afin d'assurer une meilleure gestion des projets de recherche, des données scientifiques et des documents produits.

1.2.3 Objectifs

L'objectif principal de ce mémoire est de concevoir une plateforme web de gestion des données, projets, documents et activités des chercheurs du CRA de Saint-Louis.

De manière spécifique, il s'agit de :

- Permettre à chaque chercheur de disposer d'un espace personnel pour gérer ses projets, ses tâches et ses documents ;
- Centraliser les données scientifiques et assurer leur traçabilité et leur sécurité ;
- Faciliter la collaboration entre chercheurs à travers le partage de fichiers, la planification de séminaires et le suivi des activités collectives ;
- Offrir à l'administration une vue globale des activités et indicateurs de performance scientifique ;
- Favoriser la valorisation et la diffusion des résultats de recherche produits au CRA.

1.3 Démarche Méthodologique

Pour atteindre les objectifs fixés, une démarche méthodologique rigoureuse a été adoptée, articulée autour de deux axes : la gestion de projet et l'analyse des besoins.

1.3.1 Méthode de gestion de projet

Le projet a été conduit suivant une approche agile inspirée de la méthode *Scrum*, qui favorise la flexibilité et l'adaptation progressive du produit aux besoins des utilisateurs. Cette approche se justifie par le caractère évolutif du projet et la nécessité de tester rapidement les fonctionnalités développées avec les futurs utilisateurs (chercheurs et encadrants).

Les principales étapes de cette approche sont :

- L'analyse des besoins et la rédaction du cahier des charges ;
- La modélisation conceptuelle et logique du système d'information ;
- Le développement incrémental des modules de la plateforme ;
- Les tests, validations et ajustements selon les retours des utilisateurs ;
- La mise en production et la documentation technique.

1.3.2 Méthode d'analyse

L'analyse du système s'est appuyée sur des entretiens et des observations au sein du CRA de Saint-Louis afin de bien comprendre le fonctionnement des processus scientifiques et les attentes des utilisateurs. Les méthodes suivantes ont été mobilisées :

- L'analyse fonctionnelle, pour identifier les besoins métiers et les fonctionnalités clés de la plateforme ;
- L'analyse des utilisateurs, pour définir les profils et leurs rôles respectifs (chercheur, cadre, technicien, administrateur) ;
- La modélisation UML, pour représenter les cas d'utilisation, la structure des entités et leurs relations ;
- L'analyse technologique, pour sélectionner les outils et langages adaptés au développement.

Cette démarche intégrée a permis d'assurer la cohérence entre les besoins fonctionnels exprimés et les choix technologiques retenus pour la mise en œuvre du projet.

État de l’art

Introduction

L’objectif de ce chapitre est de dresser un état des lieux des travaux existants dans le domaine des systèmes d’information pour la recherche agricole, des plateformes collaboratives de gestion de données scientifiques et des technologies web modernes. Cette revue de la littérature permet de positionner le travail réalisé par rapport aux solutions existantes et de justifier les choix architecturaux et technologiques adoptés pour la conception de la plateforme du CRA de Saint-Louis.

Ce chapitre s’articule autour de cinq axes principaux : les systèmes d’information pour la recherche agricole, les plateformes collaboratives et la gestion de projets scientifiques, les technologies web modernes pour les systèmes d’information, les mécanismes de sécurité et de gestion des accès, et enfin le positionnement de la solution proposée par rapport à l’existant.

2.1 Systèmes d’information pour la recherche agricole

2.1.1 Enjeux de la gestion des données scientifiques en agriculture

La recherche agricole moderne génère des volumes considérables de données hétérogènes : données expérimentales, rapports techniques, publications scientifiques, images, données géospatiales et informations socio-économiques. La gestion efficace de ces données constitue un défi majeur pour les institutions de recherche, particulièrement dans les pays en développement où les ressources humaines et techniques sont souvent limitées.

Plusieurs enjeux caractérisent la gestion des données scientifiques dans le domaine agricole :

La dispersion et la fragmentation des données : Les données sont souvent stockées de manière décentralisée, dans des formats hétérogènes et sans standards communs, rendant leur exploitation et leur partage difficiles.

La capitalisation des connaissances : L’absence de systèmes de gestion structurés entraîne une perte importante de capital scientifique, notamment lors des départs de chercheurs ou du renouvellement

du personnel.

L'interopérabilité et le partage des données : Les collaborations scientifiques nécessitent des mécanismes permettant l'échange et l'intégration de données provenant de sources multiples et distribuées.

La traçabilité et la reproductibilité : La recherche scientifique exige une documentation rigoureuse des protocoles, des méthodes et des données pour garantir la reproductibilité des résultats.

2.1.2 Plateformes internationales de gestion de données agricoles

Plusieurs initiatives internationales ont été développées pour répondre aux défis de la gestion des données de recherche agricole.

2.1.2.1 CGIAR Platform for Big Data in Agriculture

La plateforme CGIAR (Consultative Group on International Agricultural Research) Platform for Big Data in Agriculture, lancée en 2017, constitue une référence majeure dans le domaine. Cette plateforme vise à exploiter les capacités du Big Data pour accélérer et améliorer l'impact de la recherche agricole internationale(10-12, 21).

La plateforme CGIAR s'articule autour de plusieurs modules :

Module ORGANIZE : Ce module fournit un support pour la génération, l'accès et la gestion des données selon les principes FAIR (Findable, Accessible, Interoperable, Reusable)(11).

GARDIAN (Global Agricultural Research Data Innovation & Acceleration Network) : GARDIAN est le moteur de découverte de données phare du CGIAR. Il permet la découverte de publications et de jeux de données à travers plus d'une trentaine de dépôts institutionnels de centres CGIAR et au-delà(12). En 2019, GARDIAN référençait près de 100 000 publications et 3 000 jeux de données(12).

CGLabs : Un environnement analytique et une plateforme de collaboration permettant le développement collaboratif de pipelines analytiques et la génération de jeux de données de référence de haute qualité(11).

Services partagés : La plateforme permet l'accès à des services partagés incluant le partage sécurisé de grands ensembles de données, l'imagerie satellitaire commerciale et des données météorologiques mondiales(10).

2.1.2.2 ARDN (Agricultural Research Data Network)

L'Agricultural Research Data Network (ARDN) propose une approche différente en créant un réseau distribué de données harmonisées de recherche sur les systèmes de cultures(3). ARDN fournit des outils et des protocoles permettant non seulement le partage des données, mais aussi leur interopérabilité et leur réutilisation.

L'objectif d'ARDN est de rendre ces ensembles de données disponibles à travers des portails existants tels que l'Ag Data Commons de l'USDA et GARDIAN du CGIAR(3). Les fournisseurs de données disposent d'outils pour « ARDN-iser » leurs données, les rendant ainsi réutilisables, tandis que les utilisateurs finaux peuvent télécharger et transformer les données dans des formats utilisables pour des applications de modélisation, d'analyse de données et de méta-analyses(3).

2.1.2.3 AGRIS (International System for Agricultural Science and Technology)

Depuis 1974, l'Organisation des Nations Unies pour l'alimentation et l'agriculture (FAO) fournit un support à ses pays membres pour rendre leurs résultats de recherche visibles et accessibles à travers AGRIS(15). AGRIS constitue l'un des moteurs de recherche les plus complets en littérature scientifique agricole et alimentaire, fournissant un accès gratuit à des millions de références bibliographiques dans 119 langues différentes(15).

AGRIS facilite un réseau regroupant des centaines d'organisations à travers le monde qui contribuent des connaissances et des données à la plateforme, résultant en une collection bibliographique multilingue avec une attention particulière portée aux informations scientifiques produites dans le Sud global(15).

2.1.3 Plateformes commerciales de gestion d'essais agricoles

Parallèlement aux initiatives institutionnelles, plusieurs solutions commerciales ont été développées pour la gestion des essais et des données agricoles.

Agmatix Field Trial Management Platform : Cette plateforme offre une solution complète pour la capture de données agronomiques, la gestion d'essais au champ et l'analyse de résultats(2). Elle propose des fonctionnalités avancées incluant la collecte de données hors ligne, la validation robuste des données, le stockage sécurisé et des pistes d'audit complètes(2).

Bloomeo (Doriane) : Cette plateforme aide les entreprises sans expertise interne en données à centraliser et analyser leurs données de manière rentable, en automatisant une grande partie du processus de collecte et d'analyse(28).

Ces plateformes commerciales se caractérisent généralement par une interface utilisateur intuitive, des capacités de collecte de données mobiles et hors ligne, ainsi que des outils d'analyse et de visualisation intégrés.

2.1.4 Limites des solutions existantes pour le contexte du CRA

Bien que ces plateformes internationales offrent des fonctionnalités avancées, plusieurs limitations rendent leur adoption directe difficile pour un centre de recherche comme le CRA de Saint-Louis :

Complexité et coût : Les solutions commerciales comme Agmatix nécessitent des abonnements coûteux et peuvent être surdimensionnées pour les besoins d'un centre de recherche de taille moyenne.

Adaptation au contexte local : Les plateformes internationales sont souvent conçues pour des

consortiums de recherche de grande envergure et ne répondent pas nécessairement aux spécificités organisationnelles et opérationnelles d'un centre de recherche national.

Dépendance technologique : L'utilisation de plateformes externes crée une dépendance vis-à-vis de fournisseurs externes pour la maintenance, les mises à jour et la pérennité des données.

Manque de personnalisation : Les solutions génériques ne permettent pas toujours de modéliser les processus métiers spécifiques du CRA, notamment la planification stratégique, la gestion des conventions et partenariats, ou encore les spécificités de la recherche agricole dans la vallée du fleuve Sénégal.

Connectivité et infrastructures : Certaines plateformes nécessitent une connectivité Internet stable et permanente, ce qui peut constituer une contrainte dans certaines zones d'intervention du CRA.

Ces constats justifient le développement d'une solution sur mesure, adaptée aux besoins spécifiques, aux contraintes budgétaires et aux réalités opérationnelles du CRA de Saint-Louis.

2.2 Plateformes collaboratives et gestion de projets scientifiques

2.2.1 Outils de collaboration pour la recherche scientifique

La collaboration scientifique s'appuie aujourd'hui sur une variété d'outils numériques permettant le partage de connaissances, la gestion de références bibliographiques et la coordination de projets de recherche.

2.2.1.1 Plateformes de partage et de découverte scientifique

ResearchGate : Cette plateforme sociale pour chercheurs compte plus de 20 millions de membres et permet le partage de publications, la collaboration entre chercheurs et la découverte de travaux de recherche pertinents.

Open Science Framework (OSF) : Développé par le Center for Open Science, OSF offre une infrastructure gratuite et open source pour la gestion de projets de recherche, le contrôle de versions et le partage de données(20).

Figshare : Cette plateforme permet le partage ouvert de résultats de recherche avec attribution de DOI (Digital Object Identifier), garantissant ainsi la citabilité et la pérennité des données(20).

2.2.1.2 Outils de gestion de références et de collaboration bibliographique

Zotero : Gestionnaire de références bibliographiques open source permettant la collaboration à travers des bibliothèques de groupe(5, 20). Zotero s'intègre avec les traitements de texte et facilite la citation et la génération de bibliographies.

Mendeley : Outil similaire à Zotero, proposé par Elsevier, offrant des fonctionnalités de gestion de références et de réseautage social entre chercheurs.

2.2.1.3 Plateformes d'écriture collaborative

Anara : Outil spécialisé dans l'écriture académique collaborative, permettant à plusieurs auteurs de co-écrire, éditer et suivre les modifications en temps réel(5, 26). Anara facilite le contrôle des versions et améliore les contributions des pairs.

Overleaf : Éditeur LaTeX en ligne permettant la collaboration en temps réel pour la rédaction de documents scientifiques.

2.2.1.4 Outils d'analyse qualitative collaborative

Delve : Plateforme d'analyse qualitative collaborative offrant des fonctionnalités intuitives de collaboration et des performances fiables(14). Delve se distingue par sa facilité d'utilisation et sa synchronisation en temps réel.

Dedoose : Outil d'analyse de méthodes mixtes avec des fonctionnalités de collaboration et d'accord inter-codeurs intégrées(14).

NVivo et ATLAS.ti : Logiciels d'analyse qualitative de référence offrant des fonctionnalités de collaboration, bien que leur coût et leur complexité puissent constituer des barrières pour certaines équipes de recherche.

2.2.2 Systèmes de gestion de projets adaptés à la recherche

Les projets de recherche nécessitent des outils de gestion spécifiques permettant de coordonner les activités, de suivre l'avancement et de gérer les ressources.

Redmine : Système open source de gestion de projets et de suivi d'anomalies, largement utilisé dans les environnements académiques pour sa flexibilité et ses capacités de personnalisation.

ClickUp : Plateforme de gestion de projets assistée par IA, offrant des fonctionnalités de gestion de tâches et de collaboration pour les équipes de recherche(20).

Trello et Asana : Outils de gestion de projets visuels basés sur la méthode Kanban, populaires pour leur simplicité d'utilisation et leur approche intuitive de la gestion de tâches.

Ces outils, bien qu'efficaces pour la gestion de projets génériques, ne répondent pas spécifiquement aux besoins complexes de la recherche agricole, qui nécessitent une intégration étroite entre la gestion de projets, la collecte de données scientifiques et la gestion documentaire.

2.2.3 Plateformes de gestion de données scientifiques

Au-delà des outils génériques de collaboration, plusieurs plateformes spécialisées ont été développées pour la gestion de données scientifiques.

SciPort : Plateforme web adaptable basée sur XML pour la gestion et l'intégration de données scientifiques(38). SciPort permet aux chercheurs de collecter, publier et partager facilement des données

scientifiques complexes à travers plusieurs institutions, en utilisant une architecture distribuée basée sur un serveur central(38).

SciPort se caractérise par :

- Une approche générale basée sur XML pour modéliser des données scientifiques complexes
- Une organisation hiérarchique de l'espace de données pour une navigation rapide
- Des schémas et hiérarchies personnalisables avec des définitions basées sur XML
- Une architecture permettant la gestion indépendante de documents sur des serveurs locaux
- La publication sélective de documents vers un serveur central pour une vue globale

Protocols.io : Plateforme collaborative pour la gestion de protocoles de recherche standardisés, facilitant la reproductibilité et le partage de méthodes scientifiques(20).

Zenodo : Dépôt de données de recherche open access développé par le CERN, permettant le stockage et le partage de tous types de résultats de recherche avec attribution de DOI.

2.3 Technologies web modernes pour les systèmes d'information

Le développement de systèmes d'information modernes s'appuie sur un ensemble de technologies, d'architectures et de patrons de conception éprouvés. Cette section présente une analyse comparative des principales technologies retenues pour la conception de la plateforme du CRA.

2.3.1 Architectures applicatives

2.3.1.1 Architecture 3-tiers

L'architecture 3-tiers (three-tier architecture) constitue l'un des modèles architecturaux les plus répandus pour les applications web modernes(8, 34, 36). Cette architecture divise l'application en trois couches interconnectées :

Couche de présentation (Presentation Tier) : Interface utilisateur où l'utilisateur final interagit avec le système (navigateur web, application mobile)(36). Cette couche est typiquement construite avec HTML5, CSS et JavaScript, et communique avec les autres couches via des appels d'API(34).

Couche métier (Application/Logic Tier) : Couche intermédiaire qui gère le traitement central de l'application, les règles métier et les calculs(36). Elle agit comme intermédiaire entre la couche de présentation et la couche de données.

Couche de données (Data Tier) : Gère le stockage, la récupération et la manipulation des données de l'application, utilisant typiquement une base de données relationnelle(36).

Les avantages fondamentaux de l'architecture 3-tiers résident dans la séparation claire des préoccupations(34, 36) :

Évolutivité (Scalability) : Chaque couche peut être mise à l'échelle indépendamment pour gérer une charge ou une demande accrue(36). Par exemple, il est possible d'ajouter des serveurs à la couche métier pour améliorer les capacités de traitement sans affecter l'expérience utilisateur.

Maintenabilité : Les modifications apportées à une couche ont généralement un impact minimal sur les autres, facilitant la modification, la mise à jour ou le débogage de composants spécifiques(36). Tant que les contrats entre les couches (comme les définitions d'API) ne changent pas, les développeurs bénéficient de cycles de développement plus courts et de risques réduits.

Flexibilité : Les technologies au sein des couches peuvent être mises à jour ou remplacées sans perturber l'ensemble de l'application(36).

Sécurité : La conception en couches permet des mesures de sécurité renforcées et l'isolation des données sensibles(36).

2.3.1.2 Patron de conception MVC

Le patron de conception Model-View-Controller (MVC) constitue un autre modèle architectural fondamental pour les applications web(16, 17, 33). MVC sépare l'application en trois composants logiques principaux :

Model (Modèle) : Gère les données et la logique métier. Il est responsable de la récupération, du stockage et de la manipulation des données(16).

View (Vue) : Affiche les données du modèle à l'utilisateur et envoie les entrées de l'utilisateur au contrôleur(16). Elle est passive et n'interagit pas directement avec le modèle.

Controller (Contrôleur) : Agit comme intermédiaire entre le modèle et la vue. Il gère les entrées utilisateur, met à jour le modèle en conséquence et met à jour la vue pour refléter les changements dans le modèle(16).

Le flux typique dans une architecture MVC(16) :

1. L'utilisateur interagit avec la vue (par exemple, en cliquant sur un bouton)
2. La vue reçoit l'entrée utilisateur et la transmet au contrôleur
3. Le contrôleur traite l'entrée, met à jour le modèle si nécessaire
4. Le modèle notifie la vue des changements
5. La vue demande les données au modèle pour mettre à jour l'affichage
6. Le contrôleur met à jour la vue en fonction des changements dans le modèle

Les avantages du patron MVC(17, 33) :

- **Séparation des préoccupations :** Permet une division claire entre la logique métier, la logique d'interface et la logique de contrôle
- **Développement modulaire :** Les développeurs frontend et backend peuvent travailler indépendamment

- **Réutilisabilité du code** : Les composants peuvent être réutilisés dans différentes parties de l'application
- **Testabilité** : La séparation facilite les tests unitaires de chaque composant
- **Maintenabilité** : Les modifications peuvent être apportées à un composant sans affecter les autres

De nombreux frameworks web modernes implémentent le patron MVC(33) :

- **Ruby on Rails** : A popularisé MVC dans le développement web
- **Django (Python)** : Suit un patron MTV (Model-Template-View), effectivement MVC
- **Laravel (PHP)** : Fournit MVC avec Eloquent ORM et Blade templating
- **ASP.NET Core (C#)** : Architecture MVC fortement typée pour les applications d'entreprise
- **Spring MVC (Java)** : Framework MVC de niveau entreprise

2.3.1.3 Architecture 3-tiers vs MVC : Complémentarité

Il est important de noter que l'architecture 3-tiers et le patron MVC ne sont pas mutuellement exclusifs mais complémentaires(37). L'architecture 3-tiers décrit la structuration physique du système (séparation entre serveur web, serveur d'application et serveur de base de données), tandis que MVC décrit l'organisation logique du code au sein de la couche applicative(37).

Dans la pratique, de nombreuses applications web modernes combinent ces deux approches : une architecture 3-tiers pour la structure globale du système, et un patron MVC pour l'organisation de la couche métier/applicative.

2.3.2 Technologies backend

Le backend d'une application web gère la logique métier, l'accès aux données et les interactions avec la base de données. Plusieurs technologies et frameworks sont disponibles pour le développement backend.

2.3.2.1 Node.js et Express.js

Node.js est une plateforme d'exécution JavaScript côté serveur, non bloquante et orientée événements, idéale pour créer des applications web rapides et évolutives. Node.js repose sur un modèle à thread unique basé sur une boucle événementielle (event loop), capable de gérer simultanément un grand nombre de connexions sans créer de multiples threads système. Cette architecture permet de réduire significativement la consommation mémoire tout en offrant une excellente capacité de montée en charge.

Express.js est un framework minimaliste pour Node.js qui facilite la création de routes REST, de middlewares et de contrôleurs. Express.js adopte une approche minimaliste qui laisse une grande liberté dans l'organisation du code et la structuration des services, permettant de concevoir des API performantes et facilement extensibles.

Avantages de Node.js et Express.js :

- Haute performance pour les opérations d'entrée/sortie asynchrones
- Large écosystème de packages (npm)
- JavaScript full-stack (même langage côté client et serveur)
- Adapté aux applications temps réel (WebSocket, Socket.IO)
- Communauté active et documentation abondante

Limitations :

- Moins adapté aux traitements très lourds en CPU
- Nécessite une bonne compréhension de la programmation asynchrone
- Structure d'application à définir par le développeur (peu d'opinions natives)

2.3.2.2 Alternatives backend

Django (Python) : Framework web Python de haut niveau encourageant le développement rapide et le design pragmatique. Django suit le principe « batteries included » avec de nombreuses fonctionnalités intégrées (ORM, système d'administration, authentification).

Laravel (PHP) : Framework PHP élégant et expressif, offrant une syntaxe claire et de nombreux outils pour les tâches courantes de développement web (routage, authentification, cache, sessions).

Spring Boot (Java) : Framework Java pour construire des applications et microservices, particulièrement adapté aux applications d'entreprise nécessitant robustesse et performances.

Comparaison pour le contexte du CRA :

- **Node.js/Express.js** : Optimal pour des applications nécessitant des performances temps réel, une forte réactivité et des traitements asynchrones intensifs
- **Django** : Excellent choix pour des applications nécessitant un développement rapide avec de nombreuses fonctionnalités intégrées
- **Laravel** : Adapté pour des équipes maîtrisant PHP et recherchant un framework expressif et moderne
- **Spring Boot** : Préférable pour des applications d'entreprise complexes nécessitant une architecture robuste et des performances élevées

2.3.3 ORM (Object-Relational Mapping)

Les ORM facilitent l'interaction entre le code applicatif et la base de données relationnelle en mappant les objets du code aux tables de la base de données.

2.3.3.1 Prisma ORM

Prisma se positionne comme un ORM de nouvelle génération utilisant un langage de définition de schéma (SDL) pour définir la structure de la base de données([18](#), [22](#), [25](#)). Prisma génère ensuite un client type-safe utilisé dans le code de l'application, offrant une sécurité de typage et une autocomplétion inégalées([22](#)).

Caractéristiques principales de Prisma([18](#), [22](#)) :

- **Prisma Schema Language (SDL)** : Manière déclarative de définir le schéma de base de données et les relations
- **Client type-safe** : Génère un client basé sur le schéma, offrant une sécurité de typage complète
- **Migrations intégrées** : Système de migration basé sur le schéma
- **Support multi-bases de données** : PostgreSQL, MySQL, SQLite, MongoDB, SQL Server, CockroachDB
- **Séparation des préoccupations** : Se concentre sur l'accès pur à la base de données, laissant la logique métier à l'application

Avantages de Prisma :

- Forte emphase sur la sécurité de typage
- Migrations déclaratives
- Expérience développeur exceptionnelle avec autocomplétion
- Documentation claire et complète
- Performance optimisée avec requêtes SQL générées efficacement

Limitations :

- Relativement nouveau comparé à Sequelize et TypeORM
- Moins flexible que le SQL brut pour des requêtes très complexes
- Communauté en croissance mais ressources encore limitées

2.3.3.2 Alternatives ORM

Sequelize : ORM traditionnel mature et riche en fonctionnalités pour Node.js([18](#), [22](#), [25](#)). Sequelize mappe les tables aux classes de modèles, et les instances de ces classes fournissent une interface pour les requêtes CRUD.

Caractéristiques de Sequelize([18](#), [22](#)) :

- ORM mature et bien établi avec une grande communauté
- API basée sur les promesses
- Support de PostgreSQL, MySQL, SQLite et MSSQL
- Migrations et seeders pour la gestion du schéma
- Syntaxe parfois verbeuse comparée à TypeORM et Prisma

TypeORM : ORM flexible privilégiant TypeScript, supportant les patrons Active Record et Data Mapper([18](#), [22](#)). TypeORM offre une grande flexibilité avec un support robuste des transactions.

Caractéristiques de TypeORM([18](#), [22](#)) :

- TypeScript-first avec fort typage
- Support des patrons Active Record et Data Mapper
- Relations complètes avec chargement eager et lazy
- Système de migration robuste

- Query builder puissant et type-safe
- Support de multiples bases de données

TABLE 2.1 – Tableau comparatif des ORM Node.js

Critère	Prisma	TypeORM	Sequelize
Philosophie	ORM nouvelle génération, schema-driven, client type-safe	TypeScript-first, flexible (Active Record/Data Mapper)	ORM traditionnel mature et riche en fonctionnalités
Définition schéma	Prisma SDL (fichier dédié)	Code TypeScript (décorateurs/entités)	Code JS/TS (définitions modèles)
Support TypeScript	Excellent (génération auto de types)	Excellent (natif TypeScript)	Moyen (support basique)
Courbe d'apprentissage	Douce	Modérée à forte	Modérée
Performance	Excellente (requêtes optimisées)	Bonne	Bonne
Communauté	En croissance rapide	Grande et active	Très large et mature
Cas d'usage idéal	Projets TypeScript privilégiant la sécurité de typage	Projets nécessitant flexibilité architecturale	Projets existants ou nécessitant fonctionnalités étendues

2.3.4 Bases de données relationnelles

Le choix de la base de données est crucial pour la performance, la fiabilité et la capacité d'évolution d'une application.

2.3.4.1 PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source réputé pour sa robustesse, sa fiabilité et sa conformité aux standards SQL(1, 4, 6, 23, 29).

Caractéristiques principales(4, 6, 23) :

- **Conformité SQL avancée** : Support extensif des standards SQL
- **Types de données riches** : Support de types avancés (arrays, JSON, HSTORE, types géométriques, types personnalisés)
- **MVCC natif** : Contrôle de concurrence multi-versions pour des performances optimales en lecture/écriture simultanées
- **Indexation avancée** : Support de multiples types d'index (B-tree, Hash, GIN, GiST, SP-GiST, BRIN)
- **Extensibilité** : Possibilité de créer des types de données, des fonctions et des opérateurs personnalisés
- **Transactions robustes** : Support complet des propriétés ACID

- **Sécurité renforcée** : Modèle de sécurité robuste avec sécurité au niveau des lignes, connexions chiffrées

Performances(4) :

- PostgreSQL 1,6 fois plus rapide que MySQL pour les opérations de lecture en moyenne
- PostgreSQL 6,6 fois plus rapide que MySQL pour les opérations d'écriture en moyenne
- Performances supérieures dans les scénarios de charges mixtes et intensives en écriture

Adoption(4) :

- Adoption par les développeurs : 45% (PostgreSQL) vs 41% (MySQL) en 2023
- Privilégié pour les applications d'entreprise, l'IA et les charges analytiques

2.3.4.2 MySQL

MySQL est un système de gestion de base de données open source connu pour sa rapidité et sa simplicité, particulièrement adapté aux applications web simples et aux charges intensives en lecture(1, 4, 6, 23, 29).

Caractéristiques principales(6, 23, 29) :

- **Performance en lecture** : Excellentes performances pour les requêtes simples et les applications intensives en lecture
- **Facilité d'installation et de gestion** : Configuration plus simple que PostgreSQL
- **Réplication robuste** : Fonctionnalités de réplication synchrone et asynchrone
- **Scalabilité** : Bon support de la scalabilité horizontale
- **Large adoption** : Communauté très large et ressources abondantes

Limitations(1, 6, 29) :

- Types de données moins étendus que PostgreSQL
- Support limité des types avancés (JSON moins performant que dans PostgreSQL)
- Performances moindres pour les requêtes complexes et les opérations d'écriture intensive
- Moins adapté pour les charges analytiques complexes

2.3.4.3 Choix pour le CRA de Saint-Louis

Pour le contexte du CRA de Saint-Louis, **PostgreSQL** constitue le choix optimal pour les raisons suivantes :

Nature des données : Les données de recherche agricole sont complexes et hétérogènes, nécessitant le support de types de données avancés (JSON pour les métadonnées, arrays pour les listes de valeurs, types géométriques pour les données spatiales).

Charges mixtes : Les activités de recherche génèrent à la fois des opérations de lecture (consultation de données, génération de rapports) et d'écriture intensive (collecte de données terrain, import de résultats expérimentaux).

Intégrité et fiabilité : La conformité stricte aux propriétés ACID de PostgreSQL garantit l'intégrité des données scientifiques, critiques pour la recherche.

Évolutivité : L'extensibilité de PostgreSQL permet d'adapter le système aux besoins futurs sans migration de base de données.

Adoption croissante : La tendance du marché favorise PostgreSQL, garantissant un écosystème d'outils et de support en expansion.

2.3.5 Technologies frontend

Le frontend constitue la couche de présentation de l'application, permettant aux utilisateurs d'interagir avec le système.

2.3.5.1 React.js

React.js est une bibliothèque JavaScript développée par Meta (Facebook) pour la construction d'interfaces utilisateurs interactives et performantes(24). React repose sur une architecture basée sur les composants, où chaque composant encapsule sa propre logique et son interface.

Caractéristiques principales :

- **Architecture basée sur les composants** : Permet la création de composants réutilisables et maintenables
- **Virtual DOM** : Optimise les mises à jour de l'interface en ne modifiant que les éléments nécessaires
- **Unidirectional data flow** : Flux de données unidirectionnel facilitant le débogage et la compréhension du code
- **Large écosystème** : Nombreuses bibliothèques et outils complémentaires (React Router, Redux, React Query)
- **JSX** : Syntaxe permettant d'écrire du HTML dans JavaScript
- **Hooks** : API moderne permettant d'utiliser l'état et d'autres fonctionnalités React sans classes

Avantages :

- Performance élevée grâce au Virtual DOM
- Grande communauté et ressources abondantes
- Flexibilité et liberté architecturale
- Excellent support TypeScript
- Facilite le développement d'applications progressives (PWA)

2.3.5.2 Vite

Vite est un outil de build moderne pour les applications web, offrant un démarrage rapide et un Hot Module Replacement (HMR) efficace. Développé par Evan You (créateur de Vue.js), Vite se distingue par ses performances exceptionnelles en développement.

Avantages :

- Démarrage quasi-instantané du serveur de développement
- HMR ultra-rapide pour une expérience de développement optimale
- Configuration simplifiée comparée à Webpack
- Support natif de TypeScript, JSX et CSS modules

2.3.5.3 Alternatives frontend

Angular : Framework complet développé par Google, offrant une solution batteries-included avec architecture MVC, injection de dépendances et outils intégrés. Angular convient particulièrement aux applications d'entreprise de grande envergure.

Vue.js : Framework progressif JavaScript offrant un bon équilibre entre simplicité et puissance. Vue.js est plus opinioné que React mais plus accessible qu'Angular.

Svelte : Compilateur JavaScript transformant le code en JavaScript vanilla optimisé, offrant des performances exceptionnelles et une syntaxe simple.

Pour le CRA, **React.js avec Vite et TypeScript** constitue le choix optimal, offrant un excellent équilibre entre performance, flexibilité, maintenabilité et expérience développeur.

2.4 Sécurité et gestion des accès dans les systèmes scientifiques

La sécurité constitue un aspect critique des systèmes d'information scientifiques, notamment pour protéger les données sensibles de recherche et garantir la traçabilité des actions.

2.4.1 Authentification et autorisation

2.4.1.1 JSON Web Token (JWT)

Les JSON Web Tokens (JWT) constituent une méthode populaire et efficace pour vérifier l'identité des utilisateurs et autoriser l'accès aux ressources protégées dans les applications web modernes(7, 30, 32, 35).

Structure d'un JWT : Un JWT est composé de trois parties séparées par des points(7, 32) :

1. **Header** : Contient le type de token (JWT) et l'algorithme de signature
2. **Payload** : Contient les claims (revendications) sur l'utilisateur et des métadonnées
3. **Signature** : Garantit que le token n'a pas été altéré

Flux d'authentification JWT(7, 32) :

1. L'utilisateur se connecte avec ses identifiants
2. Le serveur valide les identifiants et génère un JWT signé
3. Le client stocke le token (cookie HttpOnly recommandé)

4. Pour chaque requête authentifiée, le client inclut le JWT dans l'en-tête Authorization
5. Le serveur vérifie la signature et la validité du token
6. Si valide, le serveur traite la requête

Avantages des JWT(7) :

- **Stateless** : Toute l'information nécessaire est dans le token, pas besoin de stockage serveur
- **Scalabilité** : Facilite la scalabilité horizontale (pas de session serveur partagée)
- **Compatible multi-plateformes** : Fonctionne avec tous les langages de programmation
- **Sécurité** : Signature cryptographique, expiration automatique, claims personnalisés
- **Performance** : Pas de requête base de données pour vérifier la session à chaque requête

Considérations de sécurité(30, 35) :

Stockage sécurisé : Les JWT doivent être stockés dans des cookies HttpOnly avec les flags Secure et SameSite pour prévenir les attaques XSS et CSRF(30, 35).

Set-Cookie: auth_token=<JWT>; HttpOnly; Secure; SameSite=Strict

Révocation de tokens : L'un des défis des JWT est l'absence de mécanisme de révocation natif. Solutions possibles :

- Utiliser une liste noire (blacklist) de tokens révoqués en cache (Redis)
- Définir des durées d'expiration courtes
- Implémenter un système de refresh tokens

Protection contre les attaques(30, 35) :

- Utiliser HTTPS pour toutes les communications
- Valider et signer les tokens avec des algorithmes robustes (RS256, ES256)
- Ne jamais stocker d'informations sensibles dans le payload (qui est seulement encodé, pas chiffré)
- Implémenter une validation stricte côté serveur
- Protéger contre les attaques d'injection et de confusion d'algorithme

Alternatives à JWT :

- **Sessions serveur avec cookies HttpOnly** : Approche traditionnelle avec session stockée côté serveur
- **Opaque tokens** : Tokens opaques (identifiants aléatoires) nécessitant une recherche serveur
- **PASETO** (Platform-Agnostic Security Tokens) : Alternative à JWT avec sécurité améliorée
- **OAuth 2.0** : Framework d'autorisation pour l'accès délégué

Pour le CRA, **JWT avec cookies HttpOnly** constitue un choix approprié, offrant un bon équilibre entre sécurité, performance et simplicité d'implémentation.

2.4.1.2 Contrôle d'accès basé sur les rôles (RBAC)

Le Role-Based Access Control (RBAC) constitue un modèle de sécurité où les permissions sont associées à des rôles plutôt qu'à des utilisateurs individuels(9, 19).

Principes du RBAC :

- **Rôles** : Définition de rôles correspondant aux fonctions organisationnelles (Chercheur, Coordonnateur, Administrateur)
- **Permissions** : Définition de permissions granulaires (créer, lire, modifier, supprimer)
- **AssigNation** : Association des rôles aux utilisateurs et des permissions aux rôles
- **Hiérarchie** : Possibilité de définir une hiérarchie de rôles avec héritage de permissions

Pour le CRA, le RBAC permet de modéliser précisément les différents profils d'utilisateurs (chercheur, coordonnateur, technicien, assistant, administrateur) et leurs permissions respectives.

2.4.2 Bonnes pratiques de sécurité pour les API REST

La sécurisation des API REST nécessite l'application de plusieurs bonnes pratiques(9, 13, 19, 27, 31) :

Utiliser HTTPS systématiquement : Chiffrer toutes les communications pour prévenir l'interception et la manipulation des données(13, 19, 27).

Authentification et autorisation robustes : Implémenter des mécanismes d'authentification forts (OAuth 2.0, JWT) et un contrôle d'accès granulaire(9, 13, 19).

Validation et sanitisation des entrées : Valider et nettoyer toutes les entrées utilisateur côté serveur pour prévenir les injections SQL, XSS et autres attaques(9, 13, 19, 27).

Rate limiting : Limiter le nombre de requêtes par client dans un intervalle de temps pour prévenir les attaques par déni de service et l'abus des ressources(13, 27).

En-têtes de sécurité : Inclure des en-têtes HTTP de sécurité (Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options)(27).

Journalisation et audit : Enregistrer toutes les actions sensibles pour la traçabilité et la détection d'anomalies(13).

Versioning d'API : Implémenter un versioning dès le début pour maintenir la compatibilité (ex : /api/v1/)(13, 31).

Gestion appropriée des erreurs : Retourner des messages d'erreur significatifs avec les codes HTTP appropriés sans exposer d'informations sensibles(13, 31).

2.5 Positionnement de la solution proposée

2.5.1 Synthèse comparative des approches existantes

Le tableau 2.2 présente une synthèse comparative des principales approches et solutions existantes pour la gestion de données scientifiques en recherche agricole.

TABLE 2.2 – Synthèse comparative des approches existantes

Critère	Plateformes Inter.	Solutions Comm.	Outils Génériques	Solution CRA
Coût	Gratuit mais complexe	Coûteux (abonnements)	Gratuit mais limité	Dév. initial puis maint. interne
Adaptation contexte	Consortiums inter.	Générique secteur agri.	Non spécifique recherche	Sur mesure CRA
Personnalisation	Limitée	Faible à moyenne	Très limitée	Totale
Gestion projets	Limitée	Axée essais champ	Basique	Complète intégrée
Gestion documentaire	Axée publications	Axée données expér.	Partage fichiers	Complète structurée
Collaboration	Oui	Limitée	Oui	Oui, intégrée
Collecte données	Limitée	Excellente (mobile, offline)	Non	Oui (formulaires, offline)
Planification stratégique	Non	Non	Non	Oui (plans, axes, programmes)
Conventions partenariats	Non	Non	Non	Oui
Dépendance externe	Élevée	Élevée	Élevée	Nulle
Pérennité données	Dépend fournisseur	Dépend fournisseur	Dépend fournisseur	Contrôle total
Adaptation évolutive	Dépend fournisseur	Dépend fournisseur	Limitée	Totale

2.5.2 Justification des choix pour la solution CRA

La solution proposée pour le CRA de Saint-Louis se distingue par plusieurs choix stratégiques justifiés par les limites des approches existantes :

2.5.2.1 Architecture et technologies

Architecture 3-tiers avec patron MVC : Ce choix offre une séparation claire des préoccupations, facilitant la maintenance, l'évolutivité et permettant le travail collaboratif sur différentes couches du système(33, 36).

Stack technologique moderne (React, Node.js, Express.js, PostgreSQL, Prisma) : Cette combinaison offre :

- Performance et réactivité élevées grâce au modèle asynchrone de Node.js
- Expérience développeur optimale avec TypeScript, Prisma et Vite
- Sécurité de typage de bout en bout (frontend et backend)
- Scalabilité et capacité d'évolution
- Écosystème riche et communauté active

Programmation asynchrone : Le modèle non bloquant permet de gérer efficacement les opéra-

tions simultanées (collecte de données, génération de rapports, requêtes utilisateurs multiples) sans compromettre les performances.

2.5.2.2 Fonctionnalités spécifiques au CRA

La solution développée intègre des fonctionnalités spécifiquement conçues pour répondre aux besoins du CRA :

Planification stratégique multi-niveaux : Modélisation complète de la hiérarchie Plan stratégique → Axes → Sous-axes → Programmes → Thèmes, permettant l'alignement des projets et activités sur la stratégie institutionnelle.

Gestion intégrée des conventions et partenariats : Suivi des conventions bilatérales et multilatérales, gestion des partenaires et des contributions, traçabilité des financements.

Formulaires dynamiques de collecte de données : Création de formulaires personnalisés avec validation, support du mode hors ligne et synchronisation automatique, particulièrement adapté aux missions de terrain.

Gestion documentaire structurée : Organisation hiérarchique des documents par type (rapports, fiches d'activité, fiches techniques, données expérimentales, publications), avec versioning, partage et contrôle d'accès granulaire.

Traçabilité et audit complets : Enregistrement systématique de toutes les actions utilisateurs, garantissant la transparence et facilitant le suivi des modifications.

Espaces personnalisés par profil : Chaque type d'utilisateur (chercheur, coordonnateur, administrateur, technicien, assistant) dispose d'un espace adapté à ses responsabilités et permissions.

2.5.2.3 Avantages par rapport aux solutions existantes

Indépendance et pérennité : La solution développée en interne garantit l'indépendance vis-à-vis de fournisseurs externes et la pérennité des données scientifiques.

Maîtrise des coûts : Après l'investissement initial de développement, les coûts se limitent à l'hébergement et à la maintenance, nettement inférieurs aux abonnements des solutions commerciales.

Adaptation continue : La maîtrise du code source permet d'adapter continuellement la plateforme aux besoins évolutifs du CRA sans dépendre de cycles de développement externes.

Propriété intellectuelle : Le CRA conserve la pleine propriété de la plateforme et peut potentiellement la partager avec d'autres centres de recherche agricole.

Formation et appropriation : Une solution développée localement facilite l'appropriation par les utilisateurs et le transfert de compétences.

Intégration future : L'architecture modulaire facilite l'intégration future avec d'autres systèmes (systèmes financiers, GED institutionnels, plateformes de publication).

2.5.3 Limites et perspectives

La solution proposée présente également certaines limites qu'il convient de reconnaître :

Ressources de développement : Le développement et la maintenance d'une solution sur mesure nécessitent des ressources humaines qualifiées et un investissement continu.

Maturité comparée aux solutions établies : Les plateformes commerciales et internationales bénéficient de nombreuses années d'optimisation et de retours d'expérience.

Interopérabilité : L'interopérabilité avec des plateformes externes nécessitera le développement d'interfaces et de mécanismes d'export standardisés.

Scalabilité à très grande échelle : Si le CRA devait multiplier par 10 ou 100 le volume de données et d'utilisateurs, des adaptations architecturales seraient nécessaires.

Ces limites constituent autant de perspectives d'évolution pour les versions futures de la plateforme.

Conclusion

Cet état de l'art a permis de dresser un panorama des solutions existantes pour la gestion de données scientifiques en recherche agricole, des plateformes collaboratives pour la recherche, et des technologies web modernes.

L'analyse des plateformes internationales (CGIAR, GARDIAN, ARDN, AGRIS) et commerciales (Agmatix, Bloomeo) révèle des solutions puissantes mais peu adaptées au contexte spécifique d'un centre de recherche national comme le CRA de Saint-Louis, notamment en termes de coût, de personnalisation et d'autonomie institutionnelle.

L'étude des technologies web modernes justifie les choix architecturaux et technologiques adoptés pour la solution CRA :

- Architecture 3-tiers avec patron MVC pour une séparation claire des préoccupations
- Stack technologique moderne (React, Node.js, Express.js, PostgreSQL, Prisma) offrant performance, sécurité de typage et expérience développeur optimale
- Modèle de programmation asynchrone adapté aux charges mixtes de lecture/écriture
- Mécanismes de sécurité robustes (JWT, RBAC, HTTPS, validation des entrées)

La solution proposée se positionne comme une alternative adaptée au contexte du CRA, offrant indépendance, maîtrise des coûts, personnalisation totale et capacité d'évolution continue, tout en intégrant des fonctionnalités spécifiques aux besoins de la recherche agricole (planification stratégique, conventions, collecte de données terrain, gestion documentaire structurée).

Les chapitres suivants détailleront les spécifications fonctionnelles et non fonctionnelles, l'analyse des besoins, la conception de la solution et sa mise en œuvre concrète.

Spécifications et Analyse des besoins

3.1 Spécifications

Cette section présente les spécifications générales du système à développer. L'objectif est de décrire de manière précise et formelle l'ensemble des fonctionnalités attendues, ainsi que les contraintes techniques, organisationnelles et qualitatives qui encadrent la mise en œuvre de la plateforme numérique destinée au CRA de Saint-Louis.

La plateforme doit permettre une gestion centralisée, structurée et sécurisée des activités scientifiques, tout en offrant à chaque type d'utilisateur (chercheur, cadre, technicien, assistant de recherche, administrateur) un espace de travail adapté à ses responsabilités. Les spécifications qui suivent constituent la base du cahier des charges fonctionnel.

3.1.1 Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent l'ensemble des services que le système doit fournir aux utilisateurs. Elles ont été identifiées à partir d'entretiens, d'observations de terrain ainsi que du cadrage établi par l'équipe du CRA.

1. Gestion des utilisateurs

Le système devra :

- Permettre à l'administrateur de créer, modifier et supprimer des comptes utilisateurs ;
- Affecter à chaque utilisateur un rôle spécifique (chercheur, coordonnateur, cadre, technicien supérieur, assistant de recherche, administrateur) ;
- Garantir à chaque rôle un ensemble de droits d'accès et de permissions personnalisées ;
- Permettre la connexion sécurisée via email et mot de passe.

2. Espace personnel de travail

Chaque utilisateur devra disposer d'un espace dédié permettant :

- L'accès à un tableau de bord personnalisé ;
- La consultation des projets, activités, tâches, documents et données auxquels il est associé ;
- La gestion de son profil et de ses informations personnelles.

3. Gestion des projets et activités de recherche

Le système devra permettre :

- La création, modification et archivage de projets scientifiques ;
- L'ajout et la mise à jour des fiches d'activités liées à chaque projet ;
- L'assignation de tâches à des assistants de recherche ou techniciens ;
- Le suivi de l'avancement des projets et des activités.

4. Gestion des documents et données scientifiques

La plateforme devra :

- Autoriser le téléversement, le classement, la consultation et la suppression de documents (PDF, Word, Excel, CSV, images) ;
- Permettre l'importation et l'exportation de données au format CSV ou XLSX ;
- Garantir la traçabilité des documents publiés (rapports, fiches d'activités, fiches techniques, données expérimentales).

5. Outils de collaboration

La plateforme doit inclure :

- Un module de planification et gestion des séminaires (calendrier, invitations, participation) ;
- Le partage de documents entre utilisateurs ;
- Des fonctionnalités de commentaires sur les projets, activités ou tâches.

6. Tableaux de bord et statistiques

Le système devra :

- Offrir aux chercheurs une visualisation des données collectées (courbes, diagrammes, indicateurs) ;
- Fournir à l'administrateur une vue globale de l'ensemble des ressources du système (projets, activités, documents, données, utilisateurs).

7. Génération de rapports

La plateforme doit permettre :

- La génération automatisée de rapports PDF ;
- L'exportation de documents et données pour les besoins administratifs ou scientifiques.

3.1.2 Spécifications non fonctionnelles

Les spécifications non fonctionnelles définissent les exigences de qualité et les contraintes techniques auxquelles le système doit répondre pour assurer sa robustesse et sa pérennité.

1. Performance

- Le système doit répondre rapidement aux requêtes (moins de 3 secondes pour les opérations courantes) ;
- Les pages contenant des visualisations doivent s’afficher sans latence notable.

2. Sécurité

- Authentification sécurisée via JWT ;
- Chiffrement des mots de passe ;
- Gestion stricte des rôles et permissions ;
- Protection contre les attaques web courantes (XSS, CSRF, injections).

3. Scalabilité

- L’architecture doit permettre l’ajout futur de nouveaux modules ;
- La base de données doit supporter une croissance significative du volume de données.

4. Disponibilité

- Le système doit être accessible à distance 24h/24 ;
- Les sauvegardes doivent être régulières et automatisées.

5. Ergonomie et accessibilité

- Interface simple, intuitive et responsive ;
- Navigation claire, adaptée aux niveaux variés de maîtrise numérique.

6. Maintenabilité

- Le code doit être structuré et documenté selon les bonnes pratiques ;
- Le système doit être facilement extensible par les équipes techniques du CRA.

Les spécifications fonctionnelles et non fonctionnelles présentées précédemment définissent le cadre général du système à développer, tant du point de vue des fonctionnalités attendues que des exigences de qualité, de sécurité et de performance.

Cependant, ces spécifications ne peuvent être pleinement comprises sans une analyse approfondie des besoins réels des utilisateurs et du contexte organisationnel du CRA de Saint-Louis. C’est dans cette

optique que la section suivante est consacrée à l'analyse détaillée des besoins métiers, utilisateurs et institutionnels.

3.2 Analyse des besoins

L'analyse des besoins vise à identifier les attentes des utilisateurs, les processus métiers existants, les contraintes institutionnelles et les fonctionnalités indispensables pour répondre aux objectifs définis dans le chapitre précédent.

Cette analyse se fonde sur plusieurs séances d'observation au CRA, des entretiens avec des chercheurs, techniciens, cadres et administrateurs, ainsi que sur l'étude des documents produits.

1. Besoins métiers

Les acteurs du CRA ont exprimé les besoins suivants :

- Centraliser l'ensemble des documents scientifiques pour réduire la dispersion ;
- Organiser les projets et activités au sein d'un système unique et structuré ;
- Faciliter le partage de données et la collaboration entre équipes ;
- Suivre l'évolution des travaux de recherche via des tableaux de bord ;
- Garantir la traçabilité des documents et données collectées ;
- Disposer d'un espace personnel structuré.

2. Besoins des utilisateurs

Chercheurs / Coordonnateurs

- Gérer leurs projets, activités, données et documents ;
- Collaborer avec assistants et techniciens ;
- Générer et archiver des rapports ;
- Visualiser les statistiques liées à leurs travaux.

Assistants de recherche / Techniciens

- Accéder aux tâches assignées ;
- Télécharger, téléverser et consulter des documents ;
- Renseigner des données via des formulaires dynamiques.

Administrateur

- Gérer utilisateurs, rôles, documents, données, séminaires ;
- Superviser l'activité globale du CRA ;
- Accéder à toutes les statistiques.

3. Contraintes organisationnelles

- Diversité des profils utilisateurs ;
- Forte production de documents hétérogènes ;
- Besoin d'un accès à distance pour les missions ;
- Nécessité d'une solution modulaire et évolutive.

L'analyse des besoins a permis de mettre en évidence les attentes fonctionnelles des différents acteurs, les processus métiers existants ainsi que les contraintes organisationnelles propres au CRA de Saint-Louis.

Afin de traduire ces besoins de manière formelle, structurée et compréhensible, une modélisation du système s'avère nécessaire. Cette modélisation repose sur l'utilisation des diagrammes UML, qui constituent un outil de référence pour représenter les fonctionnalités, la structure et le comportement du système avant sa phase de conception technique.

3.3 Diagrammes UML

Les diagrammes UML (Unified Modeling Language) permettent de représenter de manière graphique et formelle les différents aspects du système : structure, comportement, interactions et architecture. Dans le cadre de la plateforme du CRA de Saint-Louis, plusieurs types de diagrammes ont été utilisés pour modéliser les besoins fonctionnels et l'architecture logicielle. Chaque type de diagramme UML utilisé apporte une vue complémentaire du système : les diagrammes de cas d'utilisation décrivent les interactions fonctionnelles, les diagrammes de classes modélisent la structure statique des données, les diagrammes de séquences illustrent les scénarios dynamiques, tandis que les diagrammes de composants et de déploiement permettent de visualiser l'architecture logicielle et physique de la solution.

3.3.1 Diagramme des cas d'utilisations

Le diagramme des cas d'utilisations est un diagramme comportemental permettant de représenter les interactions entre les utilisateurs du système (acteurs) et les différentes fonctionnalités offertes par la plateforme. Il permet d'identifier clairement le périmètre fonctionnel du système ainsi que les rôles et responsabilités de chaque utilisateur : chercheur, technicien, assistant de recherche, coordonnateur et administrateur.

Les cas d'utilisation ont été organisés en cinq modules principaux en raison de la complexité fonctionnelle du système : (i) Vue Globale modulaire, (ii) gestion des utilisateurs, (iii) gestion des projets, (iv) gestion des activités, (v) gestion publication et formation

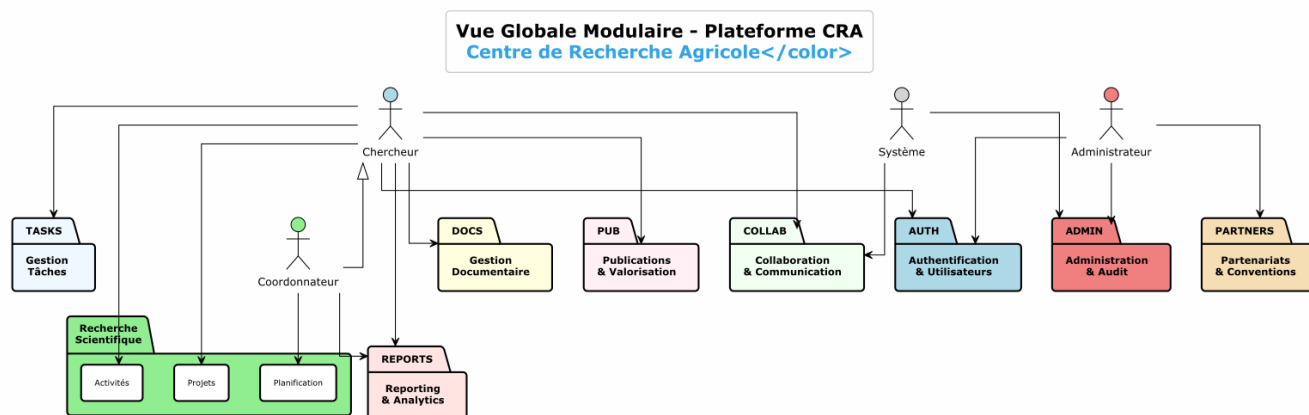


FIGURE 3.1 – Diagramme des cas d'utilisation — Vue Globale Modulaire

Description : Ce diagramme représente les différents module permettant aux chercheurs , coordonnateurs et administrateur de gérer les projets de recherche, les fiches d'activités, les tâches et le suivi scientifique.

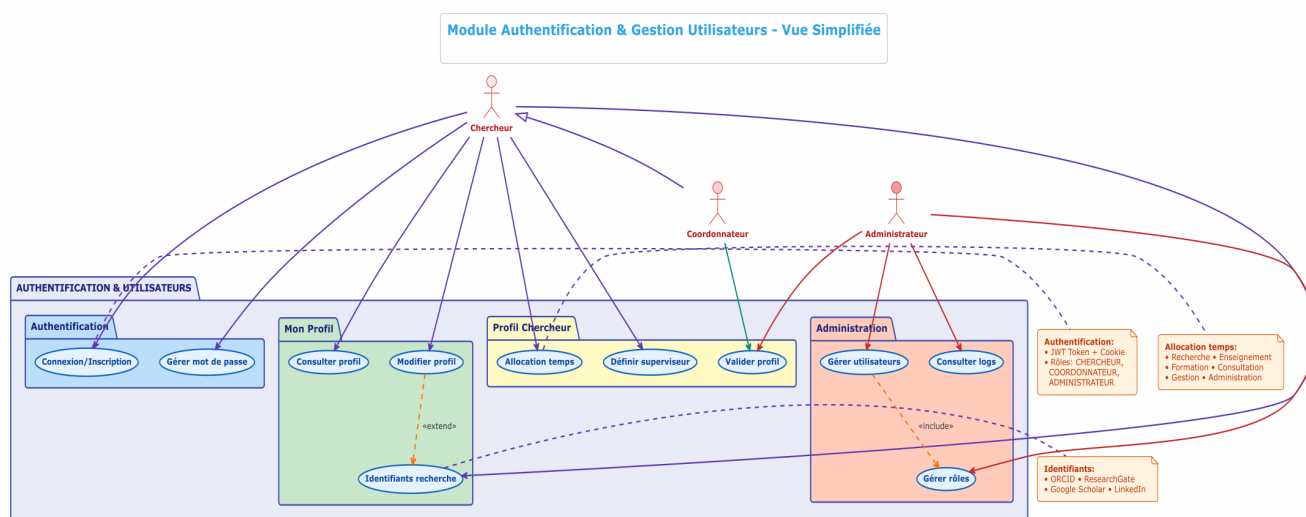


FIGURE 3.2 – Diagramme des cas d'utilisation — Module Authentification et Gestion Utilisateurs

Description : Ce diagramme illustre les interactions entre l'administrateur et la plateforme pour la gestion des comptes utilisateurs : création, modification, suppression et assignation des rôles. Il constitue un module essentiel pour la sécurité et la gestion des accès.

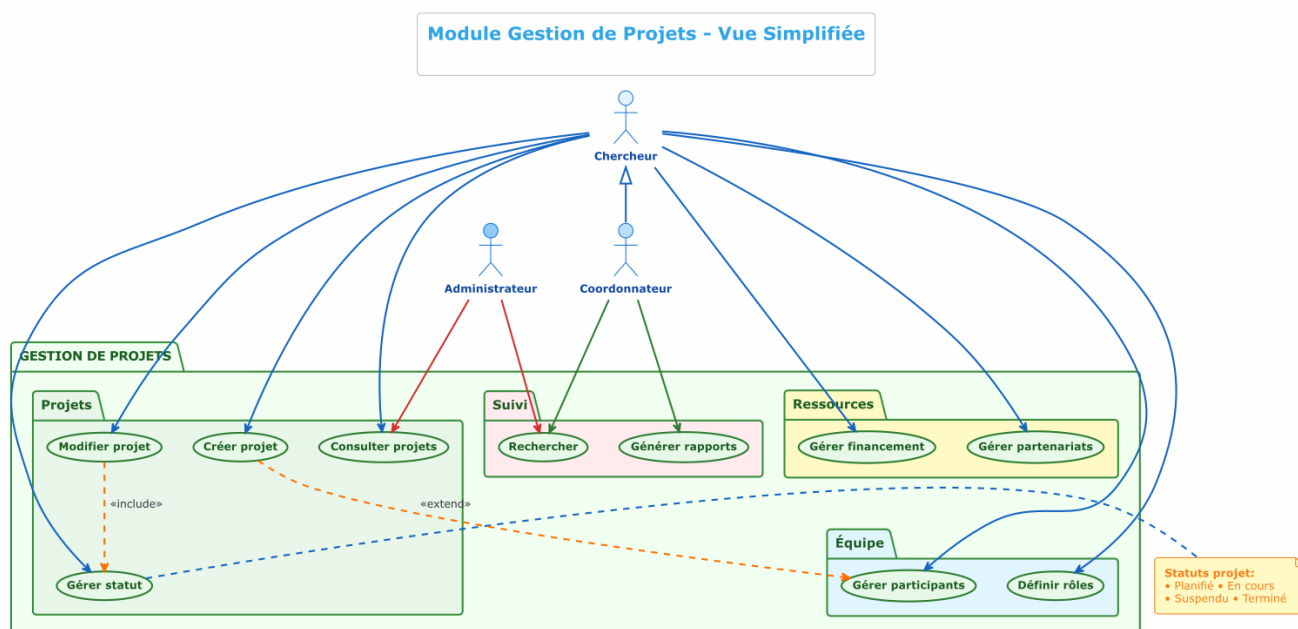


FIGURE 3.3 – Diagramme des cas d'utilisation — Module Gestion des projets

Description : Ce diagramme représente les fonctionnalités permettant aux chercheurs et coordonnateurs de gérer les projets de recherche.

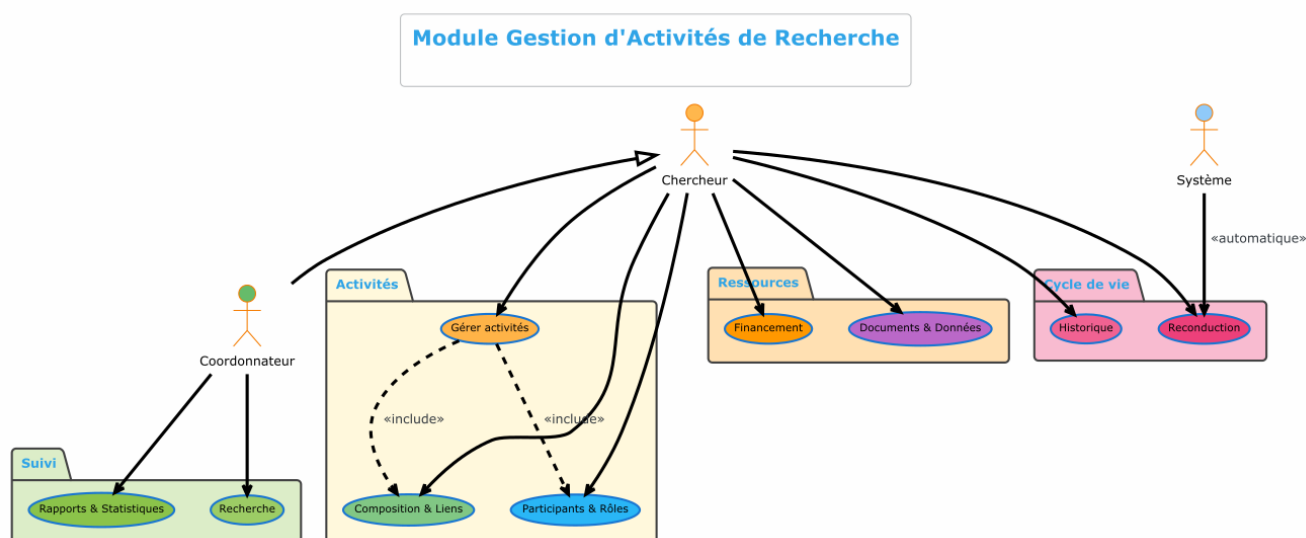


FIGURE 3.4 – Diagramme des cas d'utilisation — Module Gestion des activités

Description : Ce diagramme représente les fonctionnalités permettant aux chercheurs et coordonnateurs de gérer les fiches d'activités, les tâches et le suivi scientifique. Il met en évidence les liens entre les acteurs impliqués dans les activités de production scientifique.

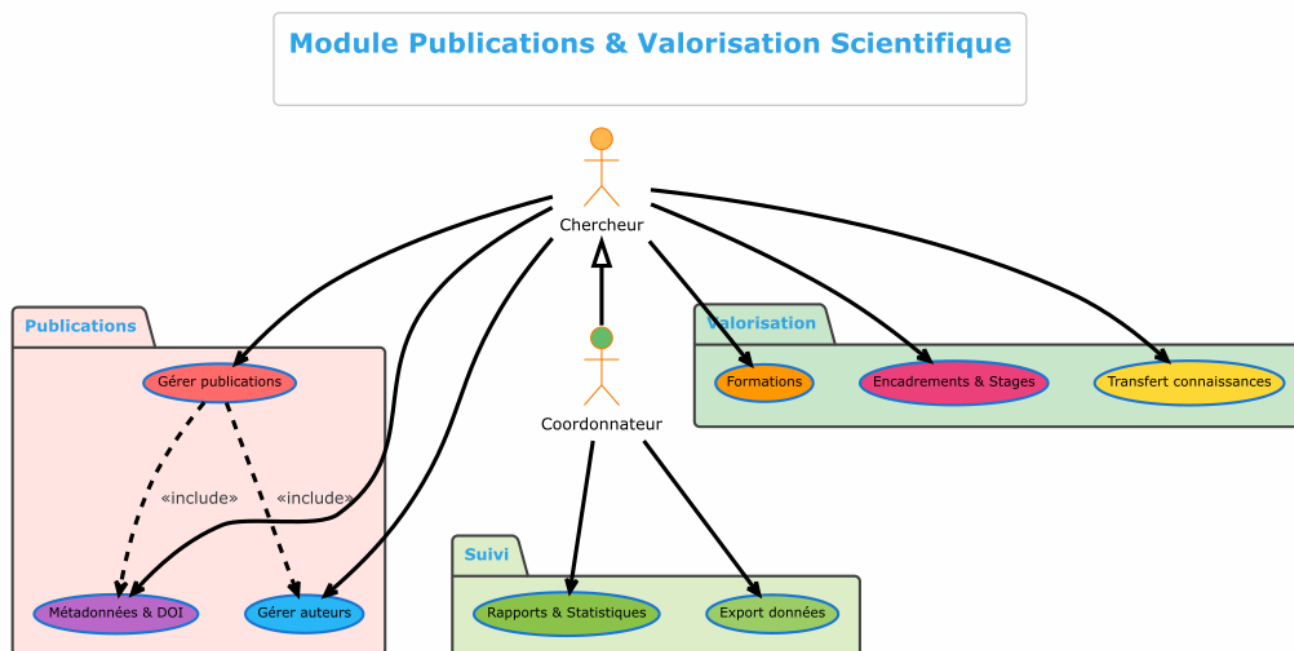


FIGURE 3.5 – Diagramme des cas d'utilisation — Module Publication

Description : Le diagramme de cas d'utilisation présenté illustre le fonctionnement du module Publications et Valorisation Scientifique de la plateforme développée pour le Centre de Recherches Agricoles (CRA) de Saint-Louis. Ce module a pour objectif principal de soutenir la production scientifique, la capitalisation des connaissances et la diffusion des résultats de recherche.

3.3.2 Diagramme de classes

Le diagramme de classes est un diagramme structurel décrivant les entités du système, leurs attributs, leurs méthodes et les relations entre elles. Il constitue la base de la modélisation du système, notamment pour la conception de la base de données et l'architecture backend.

En raison de la richesse du modèle de données de la plateforme du CRA, le diagramme de classes a été découpé en six sous-modèles : (i) utilisateurs et authentification, (ii) projets et activités, (iii) documents et données scientifiques, (iv) Gestion Documentaire (v) collaboration et séminaires, (vi) Administration et audit.

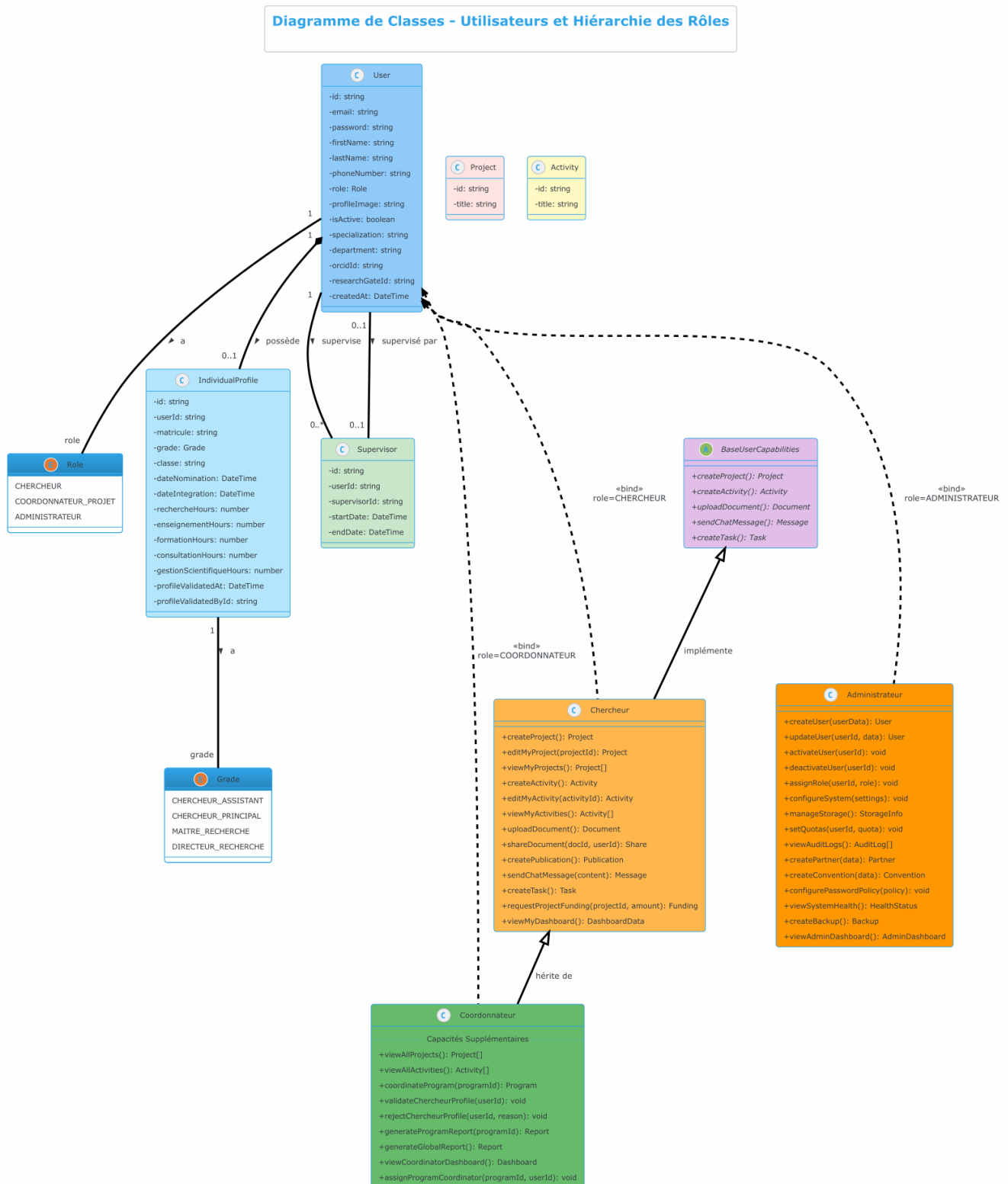


FIGURE 3.6 – Diagramme de classes — Modèle Utilisateurs et Authentification

Description : Ce diagramme présente les classes liées à la gestion des utilisateurs : **Utilisateur**, **Rôle**, **Compte** ainsi que les relations définissant les rôles, permissions et mécanismes d'authentification.

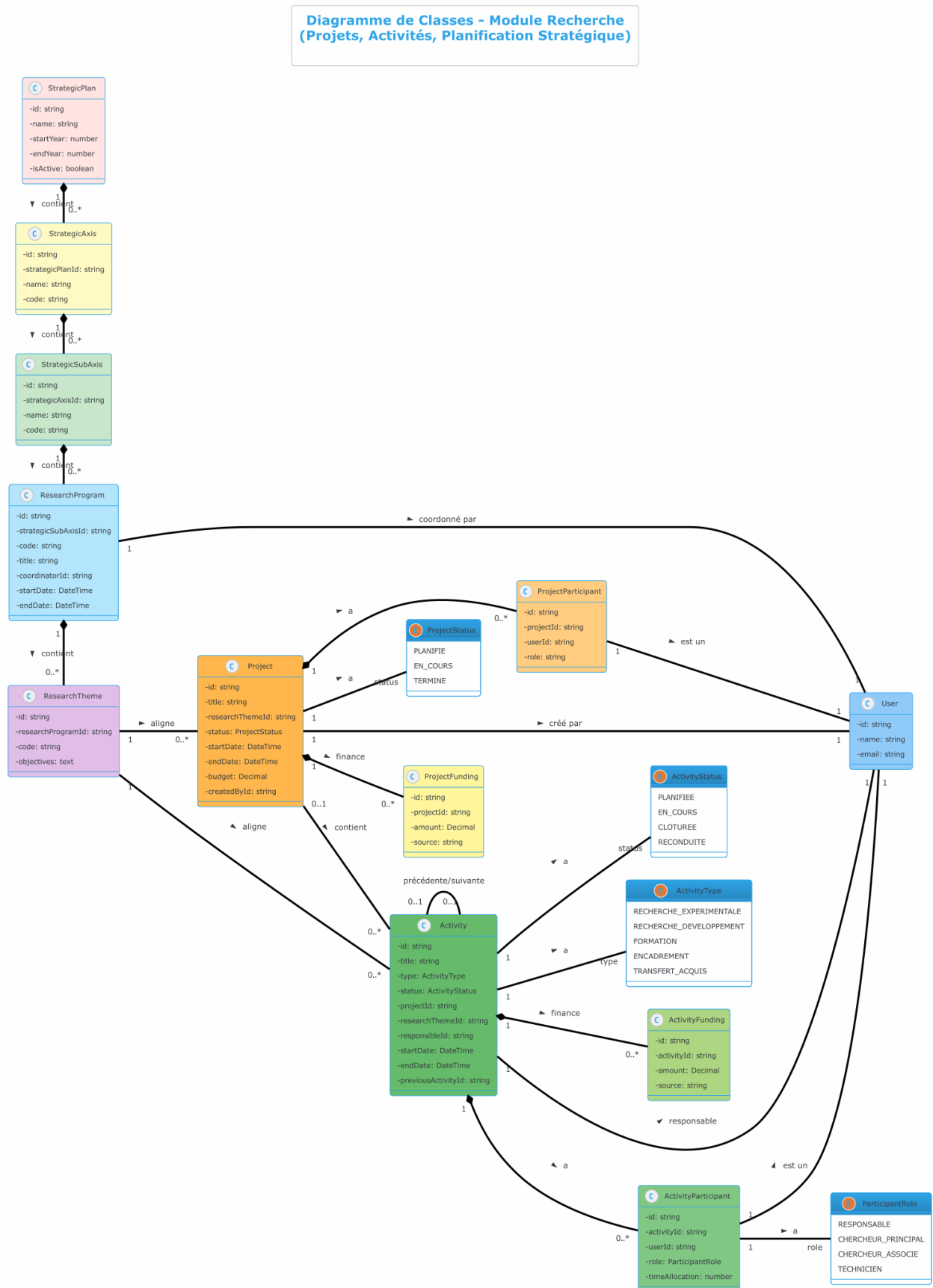
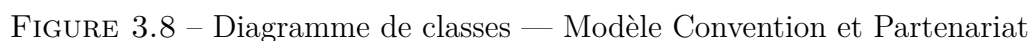


FIGURE 3.7 – Diagramme de classes — Modèle Projets et Activités

Description : Ce schéma représente la structure des entités liées aux projets de recherche : **ProjetRecherche**, **Activite**, **Tache**, et leurs relations. Il constitue le cœur du modèle scientifique de la plateforme.



Conception d'une plateforme numérique personnalisée
pour la gestion des projets et activités scientifiques du CRA

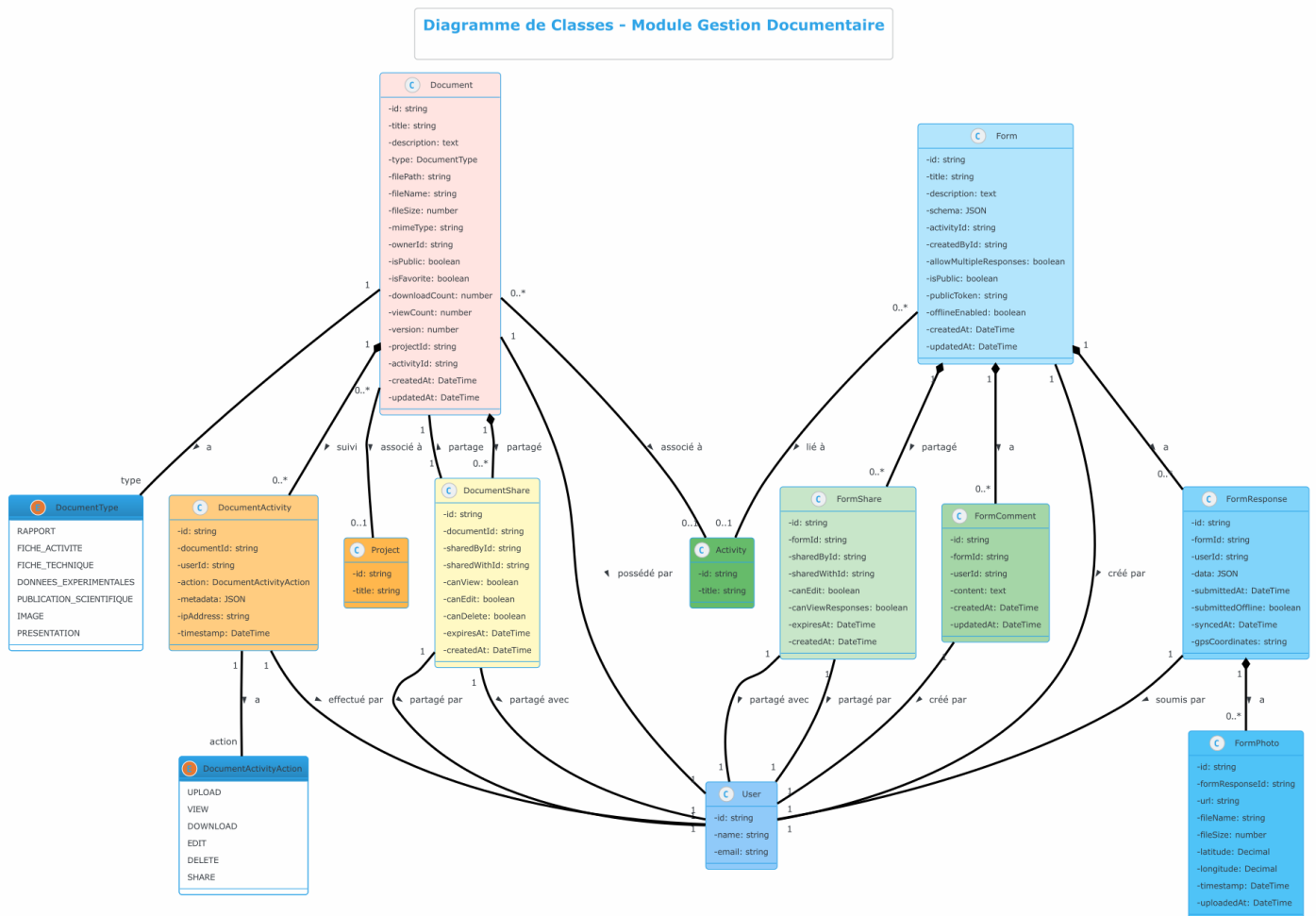


FIGURE 3.9 – Diagramme de classes — Modèle Documents et Données

Description : Ce diagramme modélise la gestion des documents, fichiers, données expérimentales et métadonnées scientifiques. Il montre les relations entre documents, projets, activités et utilisateurs.

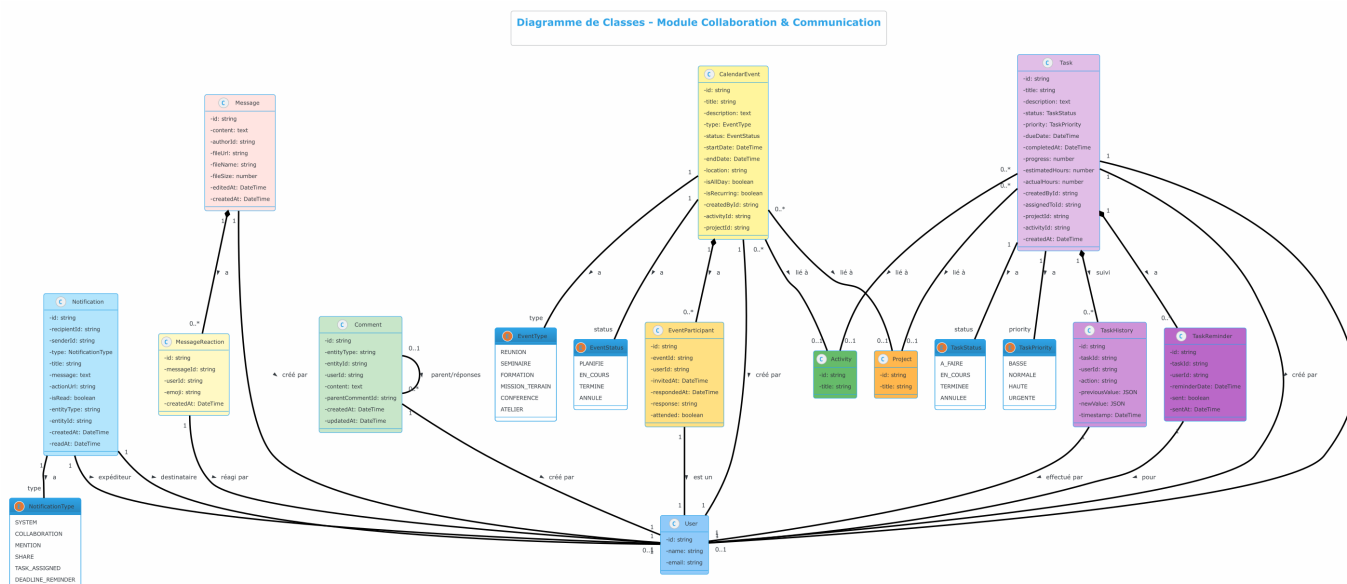


FIGURE 3.10 – Diagramme de classes — Modèle Collaboration et Séminaires

Description : Ce diagramme détaille les classes impliquées dans la planification de séminaires, la participation, les commentaires et autres interactions collaboratives entre chercheurs.

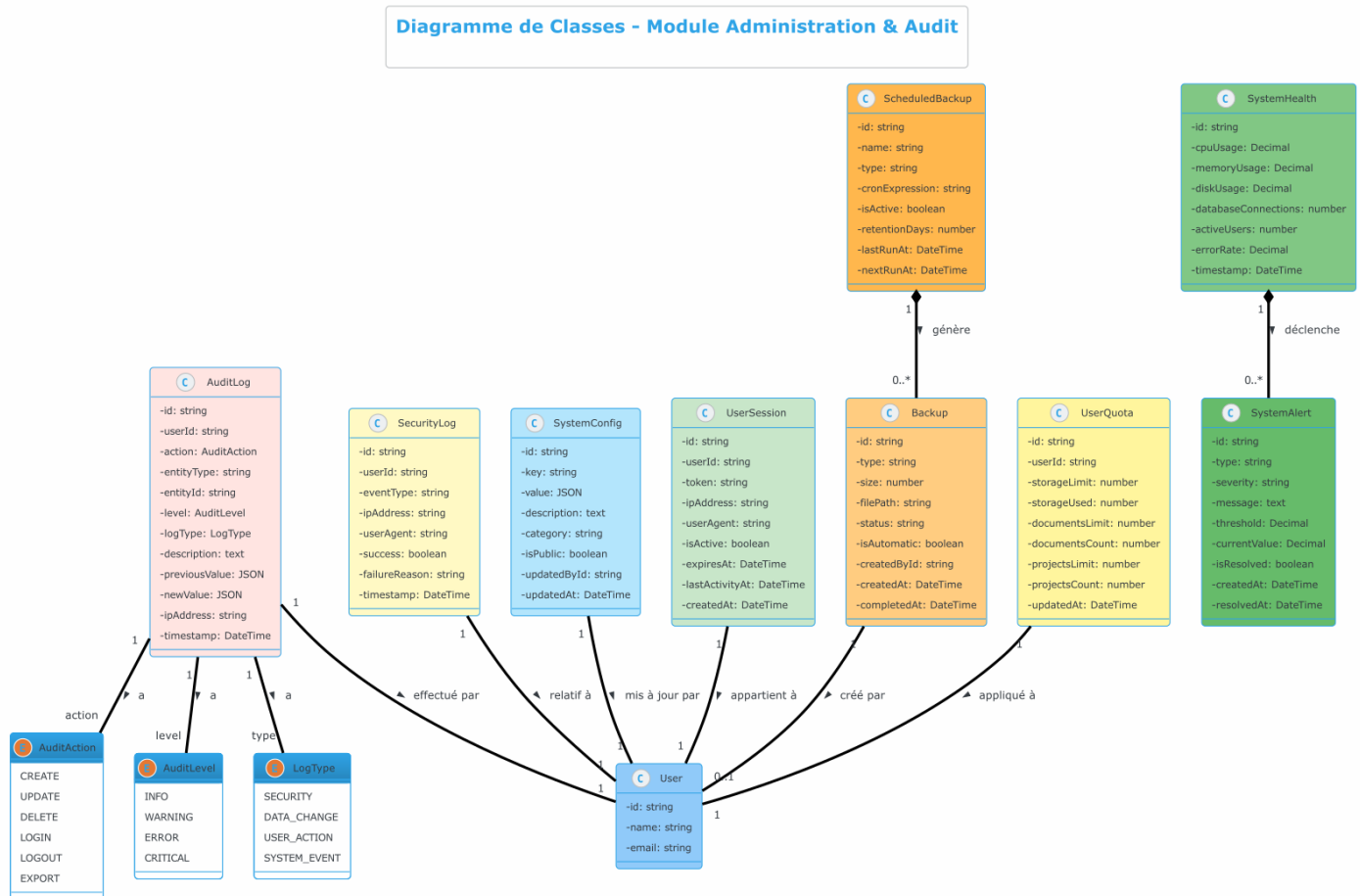


FIGURE 3.11 – Diagramme de classes — Module Administration et audit

Description : Le diagramme de classes présenté modélise l’architecture statique du module Administration et Audit de la plateforme du Centre de Recherches Agricoles (CRA) de Saint-Louis. Ce module joue un rôle central dans la gouvernance du système en assurant la sécurité, la traçabilité, la configuration, la supervision et la continuité de service.

3.3.3 Diagramme de séquences

Le diagramme de séquences est un diagramme dynamique montrant l’ordre chronologique des messages échangés entre les composants du système lors d’un scénario donné. Il permet d’illustrer le comportement opérationnel du système.

Quatre scénarios clés ont été représentés : (i) Processus de Connexion., (ii) Connection WebSocket (iii) création de projet (iv) collecte de données scientifiques

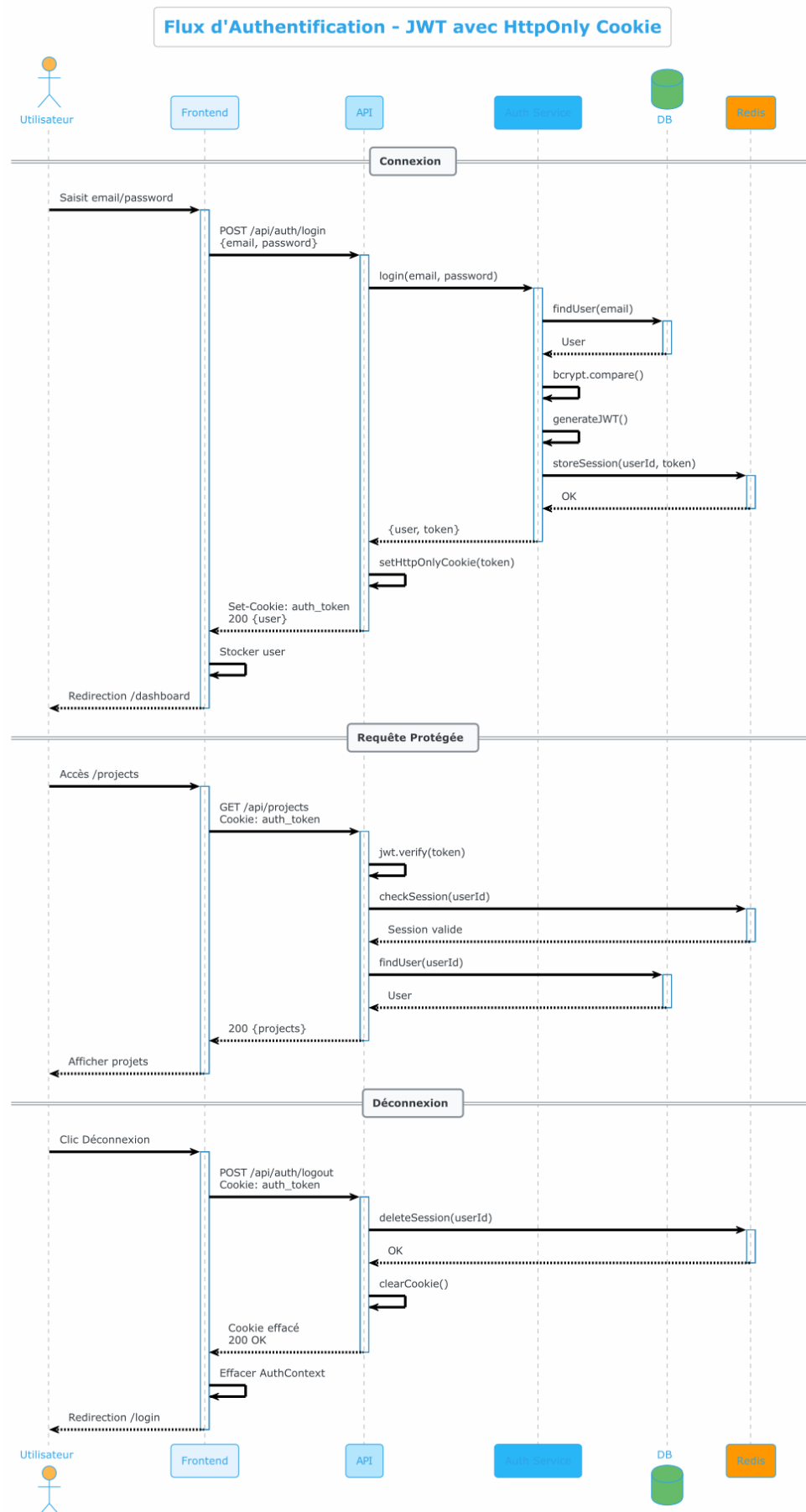


FIGURE 3.12 – Diagramme de séquence — Processus de Connexion

Description : Ce diagramme illustre les interactions entre l'utilisateur, le frontend, le backend et le module d'authentification lors de la connexion. Il met en évidence la génération et la vérification du jeton JWT.

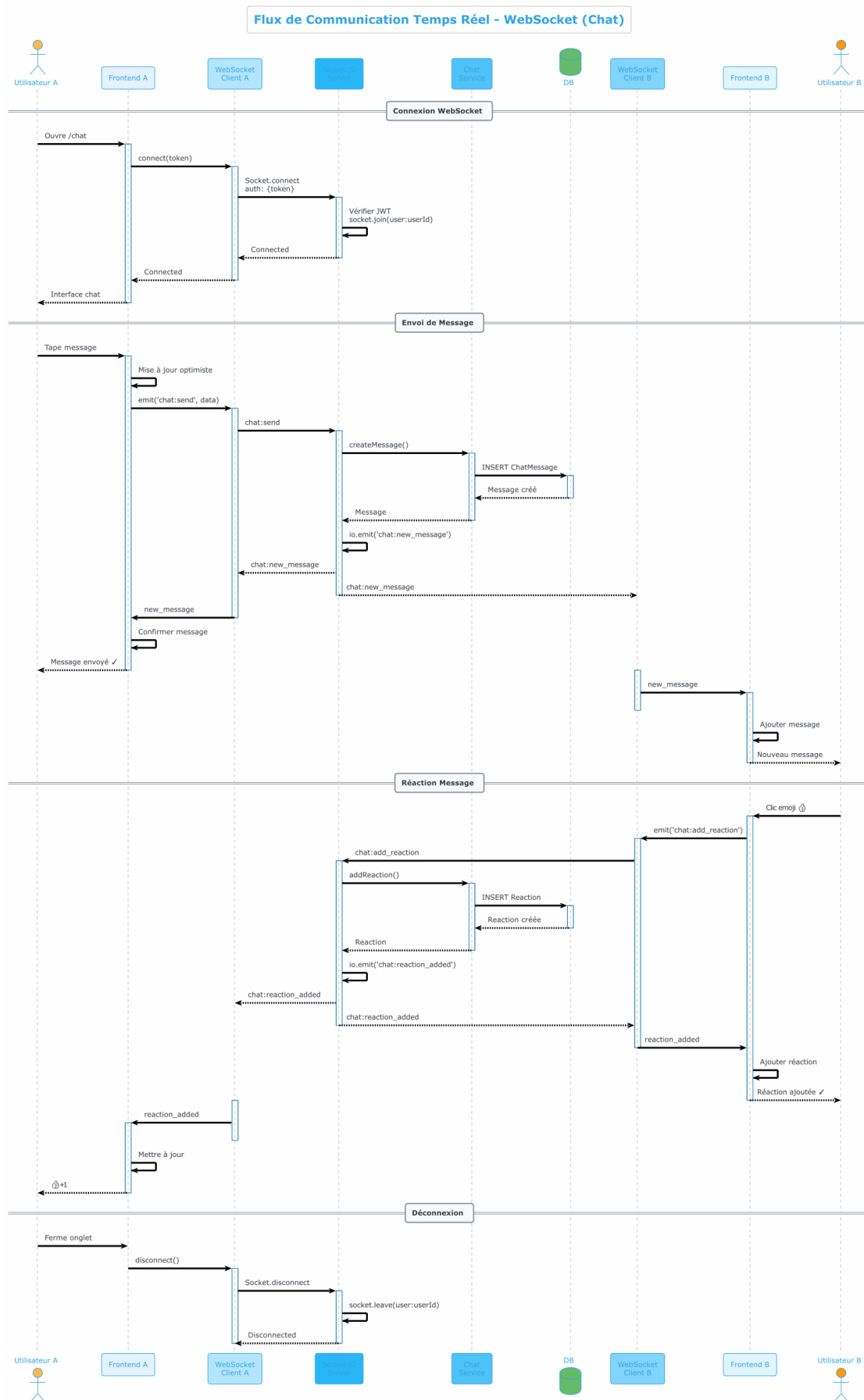


FIGURE 3.13 – Diagramme de séquence — Processus de Connexion WebSocket

Description : Ce diagramme illustre les interactions entre l'utilisateur, le frontend, le backend et le module d'authentification lors de la connexion WebSocket. Il met en évidence le processus de connexion des websockets et la vérification du jeton JWT.

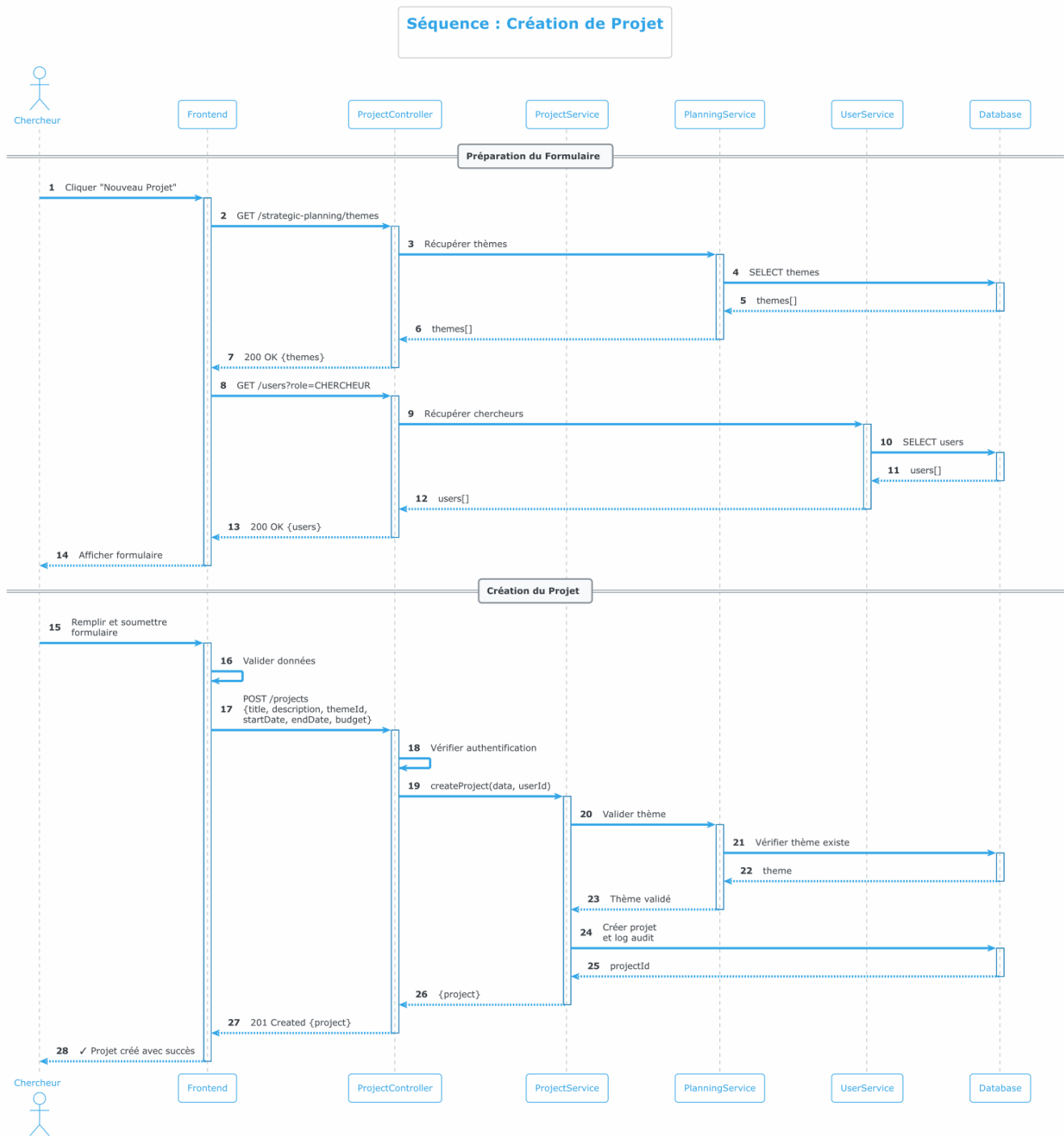


FIGURE 3.14 – Diagramme de séquence — Processus de création de projet

Description : Ce schéma montre les différentes étapes permettant à un chercheur de créer un projet : saisie, validation, enregistrement en base de données et confirmation dans l'interface.

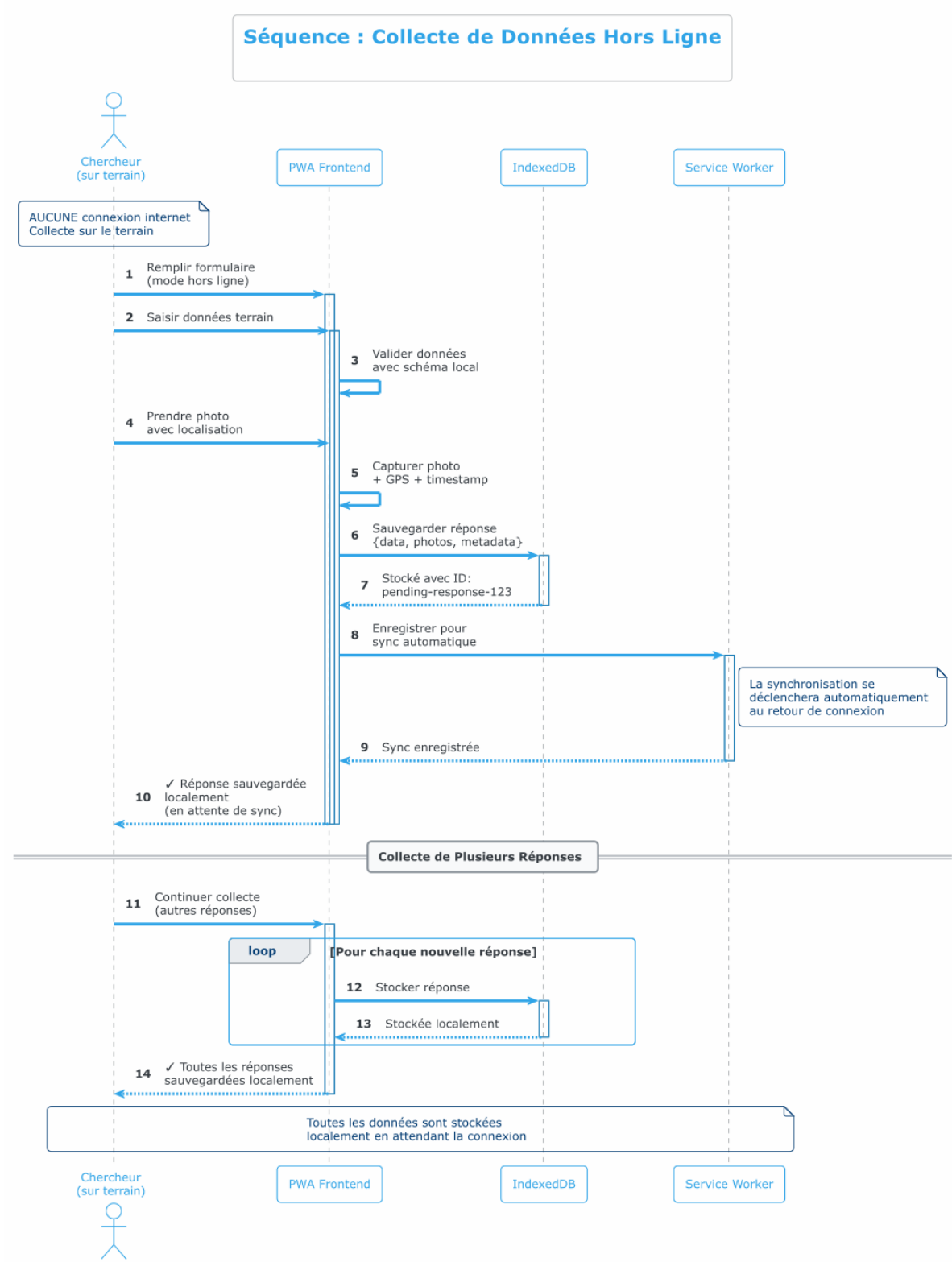


FIGURE 3.15 – Diagramme de séquence — Collete de données

Description : Ce schéma présente les interactions lors de la collecte de données via un formulaire, la validation des champs, l’enregistrement en base et l’affichage des statistiques dans le tableau de bord du chercheur.

3.3.4 Diagramme de composants

Le diagramme de composants représente l'architecture logicielle du système en illustrant les différents modules logiciels et les interactions entre eux.

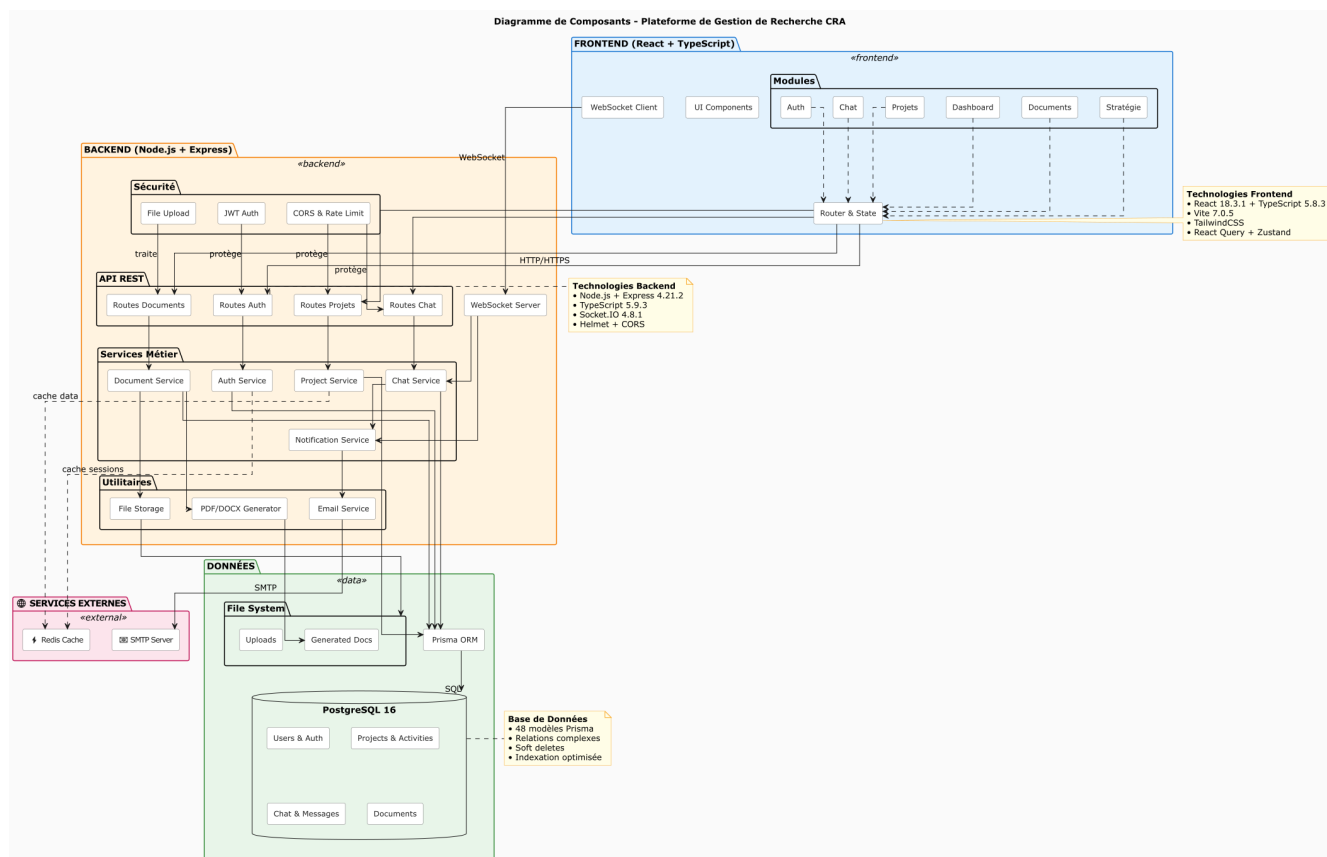


FIGURE 3.16 – Diagramme de composants — Architecture Logicielle de la Plateforme

Description : Ce diagramme montre les composants principaux de l'architecture : le frontend, l'API backend, le module JWT, le service de gestion des fichiers, les modules de statistiques et la base de données PostgreSQL.

3.3.5 Diagramme de déploiement

Le diagramme de déploiement décrit l'architecture physique du système, incluant les serveurs, les environnements et les connexions réseau.

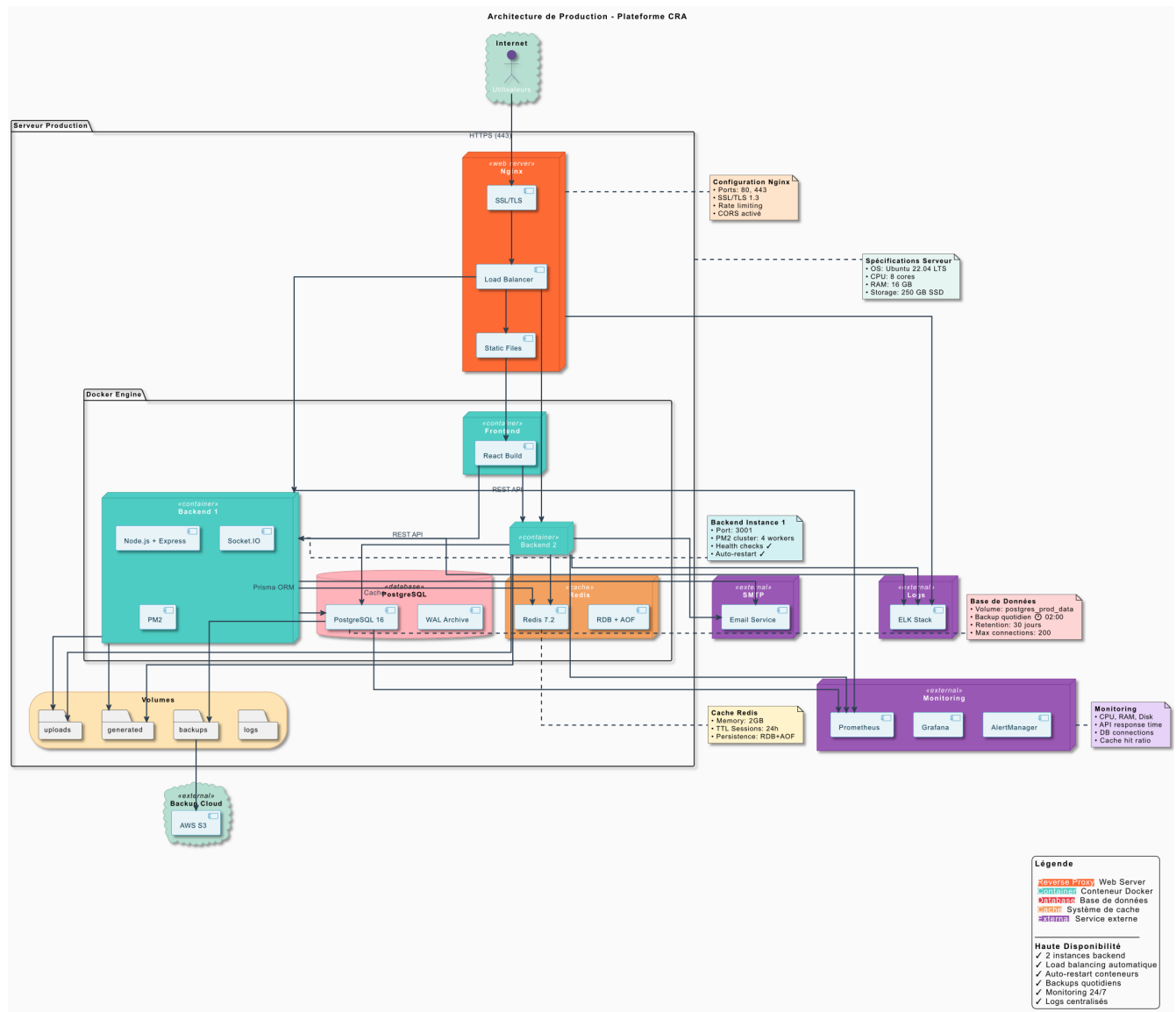


FIGURE 3.17 – Diagramme de déploiement — Architecture Physique du Système

Description : Ce diagramme illustre les nœuds physiques de déploiement : serveur web, serveur applicatif, base de données, ainsi que l'environnements de production. Il montre également le flux d'échange des données et la communication entre services.

Conclusion partielle

Ce chapitre a permis d'identifier et de formaliser l'ensemble des besoins fonctionnels et non fonctionnels de la plateforme numérique destinée au Centre de Recherches Agricoles (CRA) de Saint-Louis. À travers l'analyse des attentes des utilisateurs, des contraintes organisationnelles et des processus métiers existants, une vision claire et structurée du système cible a été établie.

La modélisation UML réalisée a joué un rôle central dans cette phase d'analyse, en offrant une représentation formelle et cohérente des fonctionnalités, des entités du système, des interactions entre les acteurs et des scénarios clés d'utilisation. Ces modèles constituent un socle essentiel pour garantir une conception maîtrisée et conforme aux exigences exprimées.

Sur la base de ces éléments, le chapitre suivant est consacré à la conception de la solution. Il présentera l'architecture globale du système, les choix technologiques retenus ainsi que la conception logique et technique de la plateforme, en s'appuyant directement sur les besoins et les modèles définis dans ce chapitre.

Conception de la solution

4.1 Architecture logique

L'architecture logique de la solution proposée repose sur une organisation modulaire et orientée services, visant à assurer la séparation des responsabilités, la maintenabilité et l'évolutivité de la plateforme. Elle définit les différentes couches fonctionnelles du système ainsi que leurs interactions, indépendamment des choix techniques et matériels.

La plateforme est structurée autour de trois grandes couches logiques :

- la couche de présentation ;
- la couche métier (ou applicative) ;
- la couche d'accès aux données.

4.1.1 Couche de présentation

La couche de présentation correspond à l'interface utilisateur de la plateforme. Elle permet aux différents profils d'utilisateurs (administrateur, chercheur, cadre, technicien, assistant) d'interagir avec le système via un navigateur web.

Cette couche assure :

- l'affichage des interfaces graphiques ;
- la saisie et la validation des données utilisateur ;
- la gestion de la navigation et des rôles ;
- la communication avec la couche métier à travers des API REST.

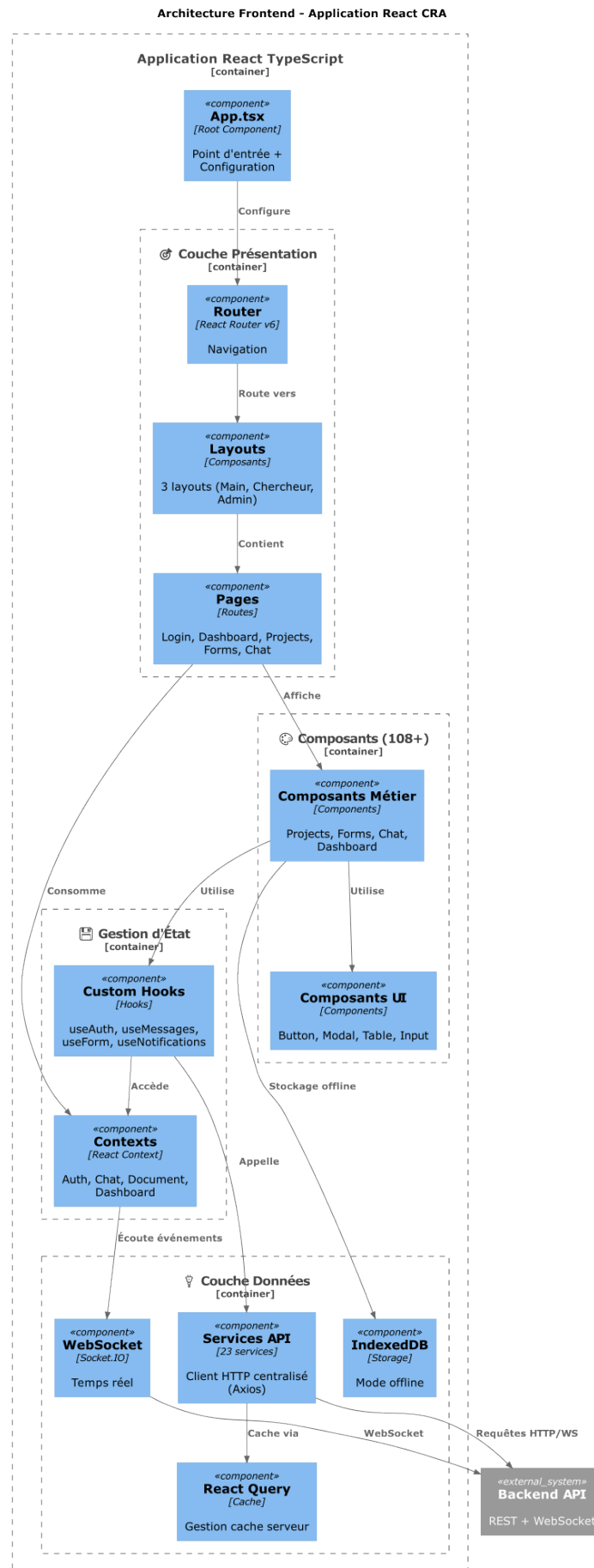


FIGURE 4.1 – Architecture de la couche de présentation de l'application CRA

Cette architecture repose sur une organisation modulaire de l'application React. Le composant `App.tsx` constitue le point d'entrée principal et assure la configuration globale de l'application. La navigation est gérée à l'aide de React Router, permettant l'accès aux différentes pages selon les routes définies.

Les interfaces sont structurées autour de plusieurs layouts adaptés aux profils des utilisateurs, tandis que les pages métiers encapsulent les fonctionnalités principales telles que la gestion des projets, des documents et des tableaux de bord. Les composants métiers s'appuient sur des composants d'interface réutilisables afin de garantir la cohérence visuelle et fonctionnelle.

La gestion de l'état applicatif est assurée par des contextes React et des hooks personnalisés, facilitant le partage des données et la communication entre composants. La couche de données s'appuie sur des services API centralisés, des mécanismes de cache via React Query et des technologies de communication temps réel telles que WebSocket, permettant ainsi une expérience utilisateur fluide et réactive.

4.1.2 Couche métier

La couche métier constitue le cœur fonctionnel de la plateforme. Elle implémente l'ensemble des règles de gestion issues de l'analyse des besoins et des spécifications fonctionnelles.

Elle est organisée en modules correspondant aux domaines fonctionnels du système, notamment :

- gestion des utilisateurs et des rôles ;
- gestion des projets et activités de recherche ;
- gestion des données scientifiques et documents ;
- publications et valorisation scientifique ;
- conventions, partenariats et financements ;
- administration, audit et sécurité.

Cette couche assure également la validation des données, le contrôle des accès et la cohérence des traitements.

4.1.3 Couche d'accès aux données

La couche d'accès aux données est responsable de la persistance et de la récupération des informations. Elle fournit une abstraction entre la couche métier et les systèmes de stockage.

Elle permet :

- la gestion des bases de données relationnelles ;
- l'accès sécurisé aux données ;
- l'optimisation des requêtes ;
- la garantie de l'intégrité et de la cohérence des données.

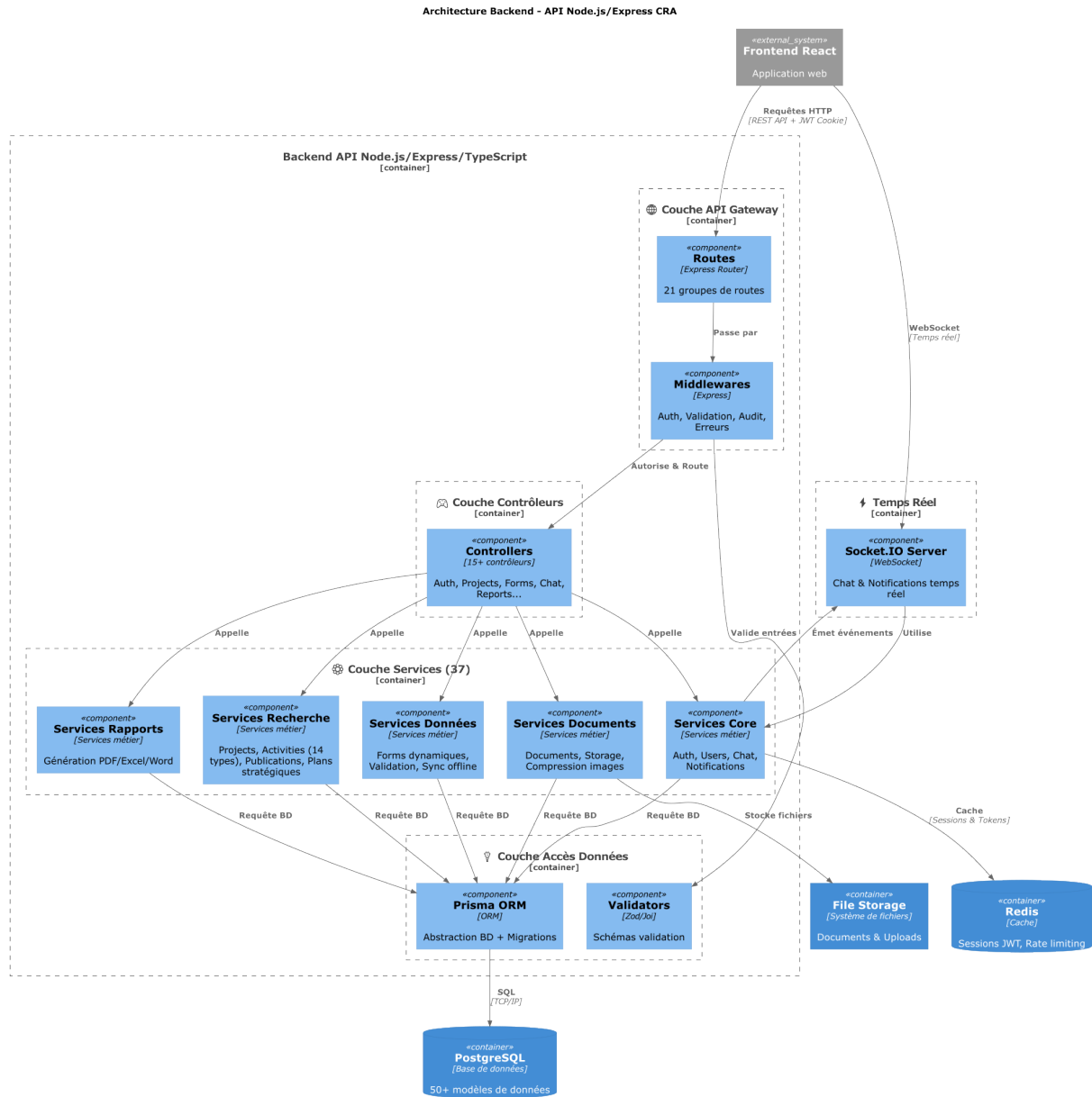


FIGURE 4.2 – Architecture de la couche métier et données – API Node.js / Express/Prisma/PostgreSQL

Comme illustré dans la figure 4.2, la couche métier est organisée selon une architecture en couches clairement séparées. Les requêtes HTTP émises par le frontend sont d’abord reçues par la couche API Gateway, composée des routes Express et des middlewares. Cette couche assure l’authentification, la validation des données, la journalisation des actions et la gestion centralisée des erreurs.

Les contrôleurs constituent le point d’entrée logique des fonctionnalités métier. Ils reçoivent les requêtes validées et délèguent le traitement aux services correspondants. Cette séparation permet de maintenir des contrôleurs légers et focalisés sur la gestion des flux de requêtes.

La couche des services regroupe l’ensemble des règles métier de l’application. Elle est subdivisée en services spécialisés, notamment les services cœur (authentification, gestion des utilisateurs, notifications), les services documents, les services données, les services recherche ainsi que les services de génération de rapports. Cette organisation favorise la modularité, la réutilisabilité du code et l’évolutivité du système.

L’accès aux données est assuré par Prisma ORM, qui fournit une abstraction typée de la base de

données PostgreSQL et gère les migrations de schéma. Des validateurs assurent la cohérence et l'intégrité des données échangées entre les différentes couches. Les mécanismes de cache et de gestion de sessions s'appuient sur Redis afin d'améliorer les performances et la scalabilité.

Enfin, la communication en temps réel est prise en charge par un serveur WebSocket basé sur Socket.IO, permettant la gestion du chat et des notifications instantanées. Les fichiers et documents sont stockés dans un système de fichiers dédié, assurant la persistance et la gestion efficace des contenus volumineux. L'architecture retenue pour cette solution repose sur un modèle 3-tiers, combinant une séparation claire entre la couche de présentation, la couche métier et la couche d'accès aux données. Par ailleurs, la couche métier s'inspire du patron de conception MVC, où les contrôleurs Express jouent le rôle de contrôleurs, les modèles Prisma représentent la couche modèle, tandis que la vue est externalisée au niveau du frontend React.

Cette architecture hybride permet de bénéficier à la fois de la rigueur du modèle MVC et de la flexibilité d'une architecture 3-tiers, tout en facilitant la maintenabilité, l'évolutivité et le déploiement indépendant des différentes couches du système.

4.2 Architecture technique

L'architecture technique décrit l'organisation physique et logicielle des composants du système ainsi que leur mode de déploiement. Elle repose sur une architecture client-serveur distribuée.

La plateforme est constituée des éléments suivants :

- un client web accessible via un navigateur ;
- un serveur applicatif hébergeant les API REST ;
- un serveur de base de données ;
- des services de sécurité et de sauvegarde.

Le client web communique avec le serveur applicatif via le protocole HTTP sécurisé (HTTPS). Le serveur applicatif interagit avec la base de données pour le stockage et la récupération des informations.

Cette architecture permet :

- une séparation claire entre les composants ;
- une meilleure scalabilité ;
- une maintenance facilitée ;
- une sécurisation des échanges.

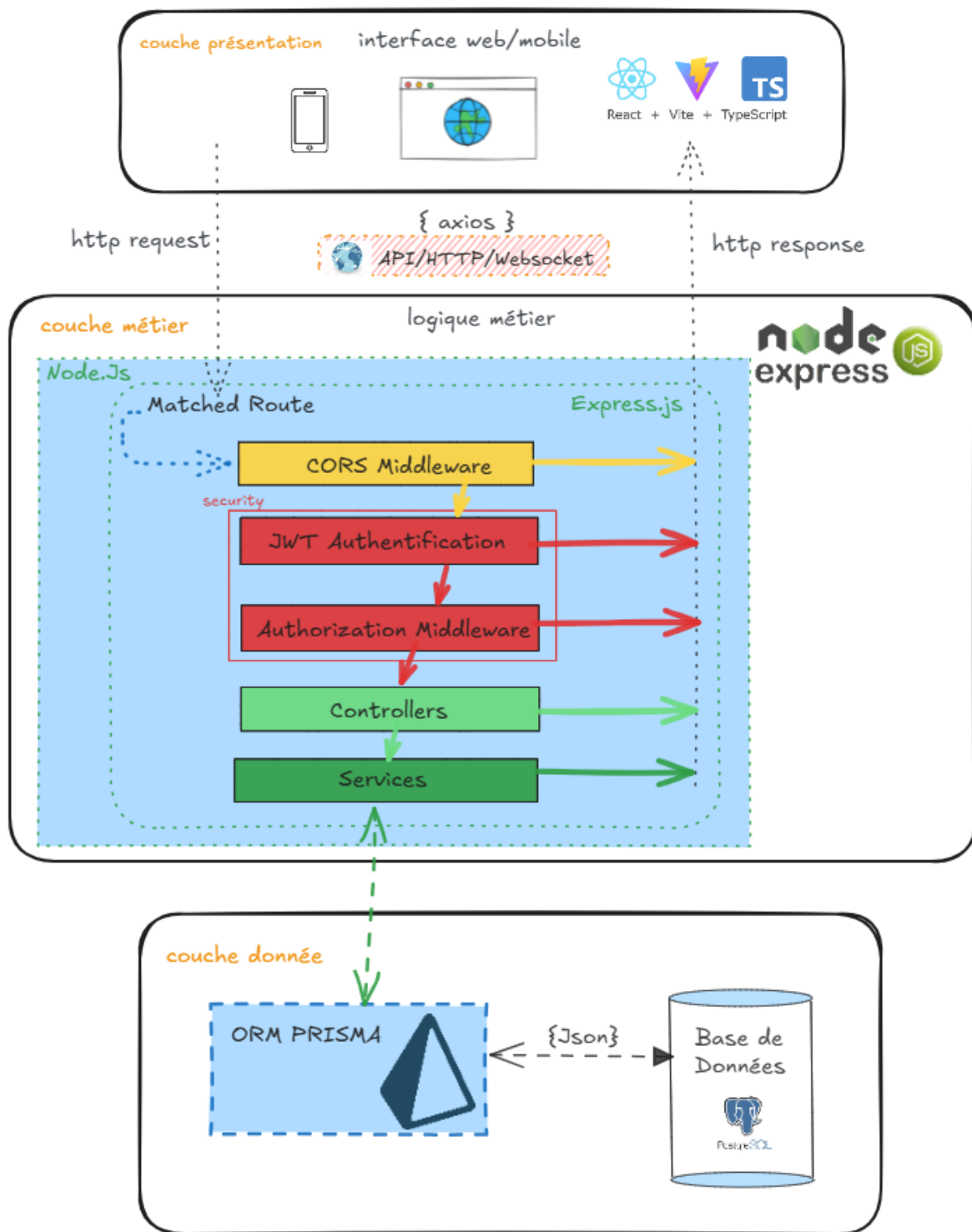


FIGURE 4.3 – Architecture technique 3-tiers de la solution (Frontend React – Backend Node.js/Express – Données PostgreSQL)

La figure 4.3 illustre l'architecture technique globale de la solution, basée sur une approche 3-tiers intégrant une communication asynchrone entre les différentes couches du système.

La couche de présentation repose sur une application web développée avec React, Vite et TypeScript. Elle constitue l'interface utilisateur accessible depuis des navigateurs web ou des terminaux mobiles. Cette couche communique avec le backend via des requêtes HTTP et WebSocket, en utilisant la bibliothèque Axios pour les appels API.

La couche métier est implémentée à l'aide de Node.js et du framework Express.js. Elle est responsable du traitement de la logique applicative et de la gestion des règles métier. Les requêtes entrantes sont d'abord interceptées par des middlewares transverses, notamment le middleware CORS pour la sécurité, le mécanisme d'authentification basé sur JWT ainsi que les middlewares d'autorisation. Une fois validées, les requêtes sont traitées par les contrôleurs Express, lesquels délèguent les traitements aux services métier.

La couche d'accès aux données repose sur l'ORM Prisma, qui assure l'abstraction entre la logique métier et la base de données PostgreSQL. Prisma facilite la gestion des modèles de données, la réalisation des requêtes SQL et l'application des migrations, tout en garantissant la cohérence et l'intégrité des données persistées.

Cette architecture technique favorise une séparation claire des responsabilités, améliore la maintenabilité du système et permet une évolutivité facilitée grâce à l'indépendance des différentes couches.

Conclusion partielle

Ce chapitre a permis de définir la conception globale de la solution proposée pour le Centre de Recherches Agricoles (CRA) de Saint-Louis. À travers la présentation de l'architecture logique, des choix technologiques et de l'architecture technique, une vision claire et structurée du système a été établie.

L'architecture retenue, fondée sur un modèle 3-tiers et inspirée du patron de conception MVC, garantit une séparation nette des responsabilités entre les différentes couches du système. Les choix technologiques opérés, tant au niveau du frontend que du backend, répondent aux exigences de performance, de sécurité, de maintenabilité et d'évolutivité identifiées lors de l'analyse des besoins.

Sur la base de cette conception, le chapitre suivant est consacré à la mise en œuvre concrète de la solution. Il détaillera l'environnement de développement adopté, les étapes de réalisation de la plateforme ainsi que l'analyse des résultats obtenus, incluant une étude des performances associées à la solution développée.

Mise en oeuvre et résultats

5.1 Environnement de développement

La mise en œuvre de la plateforme du Centre de Recherches Agricoles (CRA) de Saint-Louis a été réalisée dans un environnement de développement moderne, collaboratif et orienté qualité. Les outils sélectionnés visent à faciliter le développement, la collaboration entre les membres de l'équipe, la reproductibilité des environnements et la documentation des services exposés.

Visual Studio Code

L'environnement de développement intégré (IDE) **Visual Studio Code (VS Code)** a été utilisé pour l'ensemble du développement frontend et backend. Cet outil offre un excellent support pour les technologies employées dans le projet, notamment TypeScript, React, Node.js et Prisma.

VS Code fournit des fonctionnalités avancées telles que la complétion automatique, le débogage intégré, la gestion des extensions et la navigation efficace dans le code. Ces fonctionnalités contribuent à améliorer la productivité, la lisibilité du code et la détection précoce des erreurs.

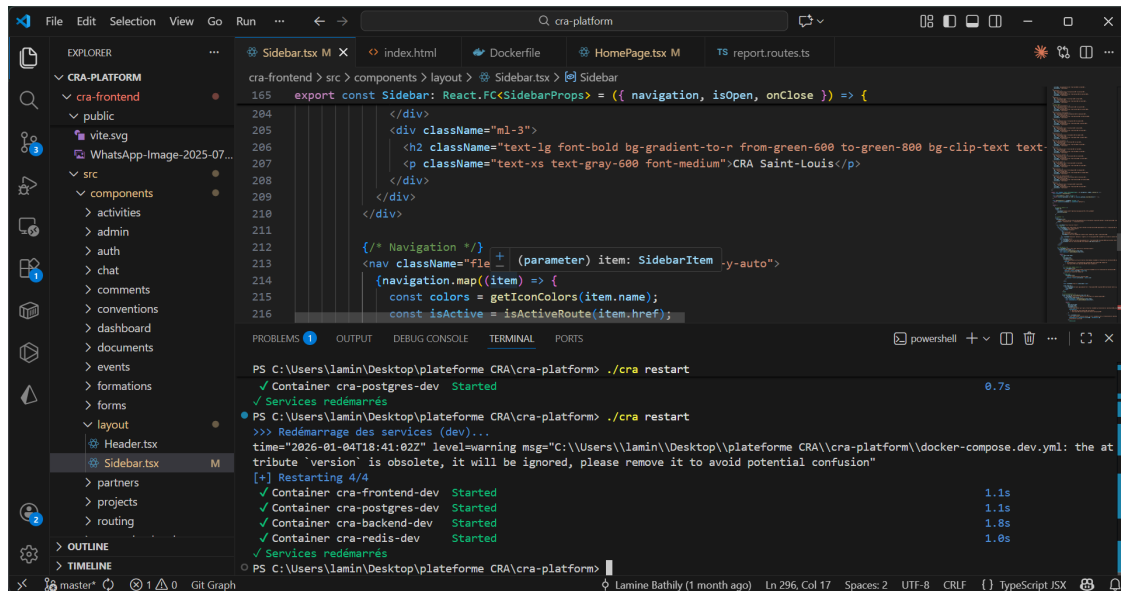


FIGURE 5.1 – Environnement de développement Visual Studio Code utilisé pour le projet

Git et GitHub

Le système de gestion de versions **Git** a été utilisé pour suivre l'évolution du code source et gérer les différentes versions du projet. Il permet de conserver un historique complet des modifications, de faciliter le travail collaboratif et de sécuriser le code contre les pertes accidentelles.

La plateforme **GitHub** a servi de dépôt centralisé pour l'hébergement du code source. Elle a également permis la gestion des branches, le suivi des issues et la revue de code, favorisant ainsi une collaboration structurée et transparente.

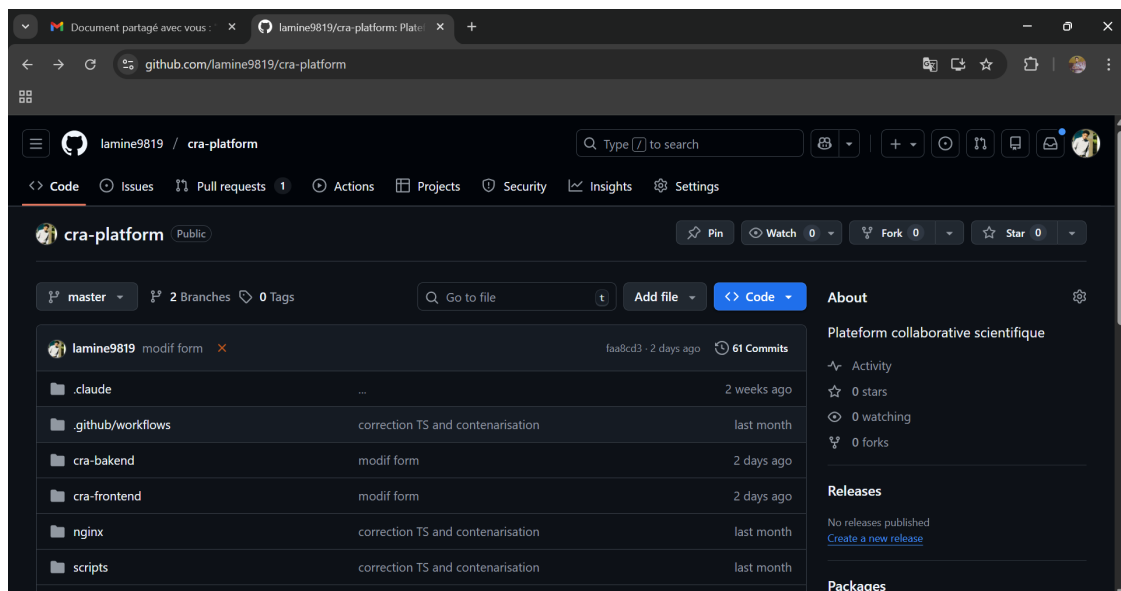


FIGURE 5.2 – Dépôt GitHub du projet et historique des versions

Docker

La technologie **Docker** a été utilisée pour la conteneurisation des différents composants de la plateforme, notamment le backend, la base de données PostgreSQL et les services associés. Docker permet de garantir la cohérence des environnements de développement, de test et de déploiement.

Grâce à Docker, l’application peut être exécutée de manière identique sur différentes machines, réduisant ainsi les problèmes liés aux dépendances et aux configurations spécifiques. Cette approche facilite également le déploiement futur et la scalabilité du système.

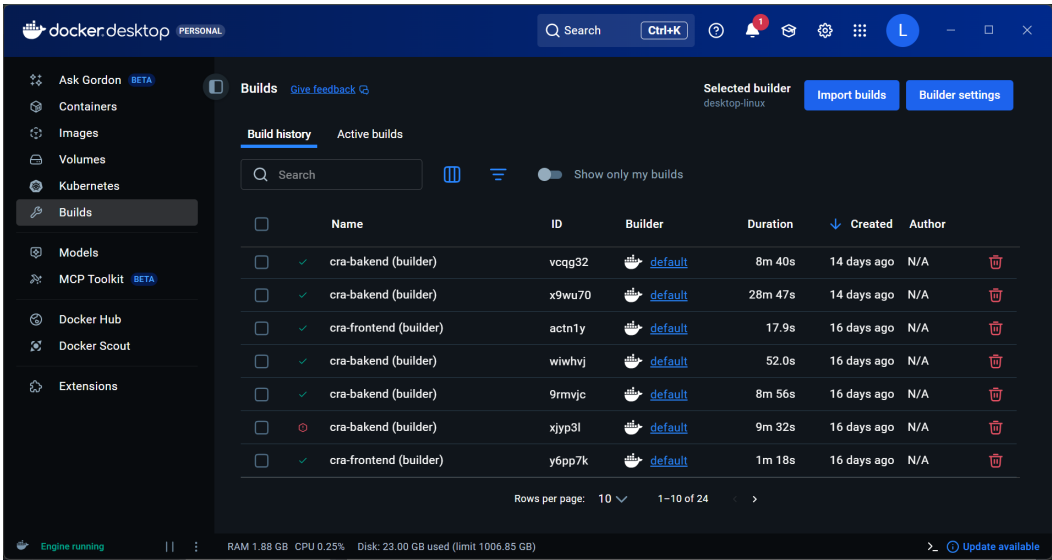


FIGURE 5.3 – Conteneurisation de l’application à l’aide de Docker

Swagger (OpenAPI)

La documentation des API REST a été réalisée à l’aide de **Swagger**, basé sur la spécification **OpenAPI**. Swagger permet de décrire de manière formelle les endpoints exposés par le backend, les paramètres attendus, les réponses retournées et les mécanismes de sécurité.

L’interface Swagger offre également la possibilité de tester les API directement depuis un navigateur, facilitant ainsi la validation fonctionnelle, le débogage et la communication entre le frontend et backend.

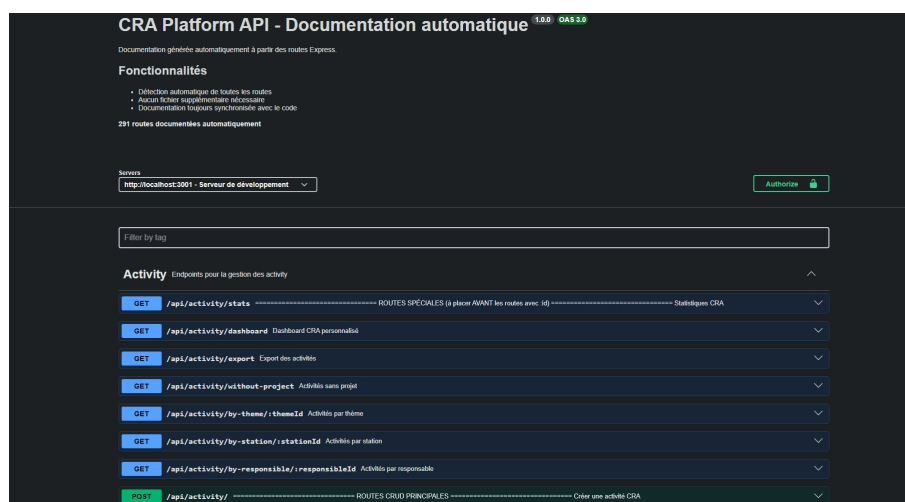


FIGURE 5.4 – Documentation interactive des API REST via Swagger

5.2 Présentation et analyse des résultats

Cette section présente les résultats obtenus suite à la mise en œuvre de la plateforme du Centre de Recherches Agricoles (CRA) de Saint-Louis. Elle s'appuie sur des captures d'écran illustrant les principales fonctionnalités développées, en mettant en évidence les usages métiers destinés aux chercheurs ainsi que les fonctionnalités administratives réservées à l'administrateur. L'introduction du rôle de coordonnateur de projet constitue un apport majeur de la solution proposée. En offrant une vue transversale sur les projets et les activités, ainsi que des outils de génération de rapports automatisés, la plateforme répond aux besoins de coordination, de suivi et de pilotage scientifique du CRA. Cette approche renforce la structuration des projets de recherche et facilite la communication entre chercheurs, coordonnateurs et instances administratives.

5.2.1 Page d'accueil de la plateforme

Avant toute authentification, l'utilisateur accède à la page d'accueil de la plateforme (figure 5.5). Cette page constitue le point d'entrée du système et présente le contexte institutionnel du CRA, les objectifs de la plateforme ainsi que les principaux domaines d'intervention.

La page d'accueil permet également d'accéder à l'interface d'authentification. Son design clair et structuré favorise une prise en main intuitive et renforce la visibilité des services offerts par la plateforme.



FIGURE 5.5 – Page d’accueil de la plateforme CRA

5.2.2 Interface d’authentification

La figure 5.6 illustre l’interface d’authentification permettant aux utilisateurs de se connecter à la plateforme à l’aide de leurs identifiants. Le mécanisme de connexion repose sur un système sécurisé basé sur des jetons JWT.

Une fois authentifié, chaque utilisateur est automatiquement redirigé vers un espace de travail adapté à son rôle, garantissant ainsi une gestion fine des droits d’accès et une séparation claire des responsabilités.

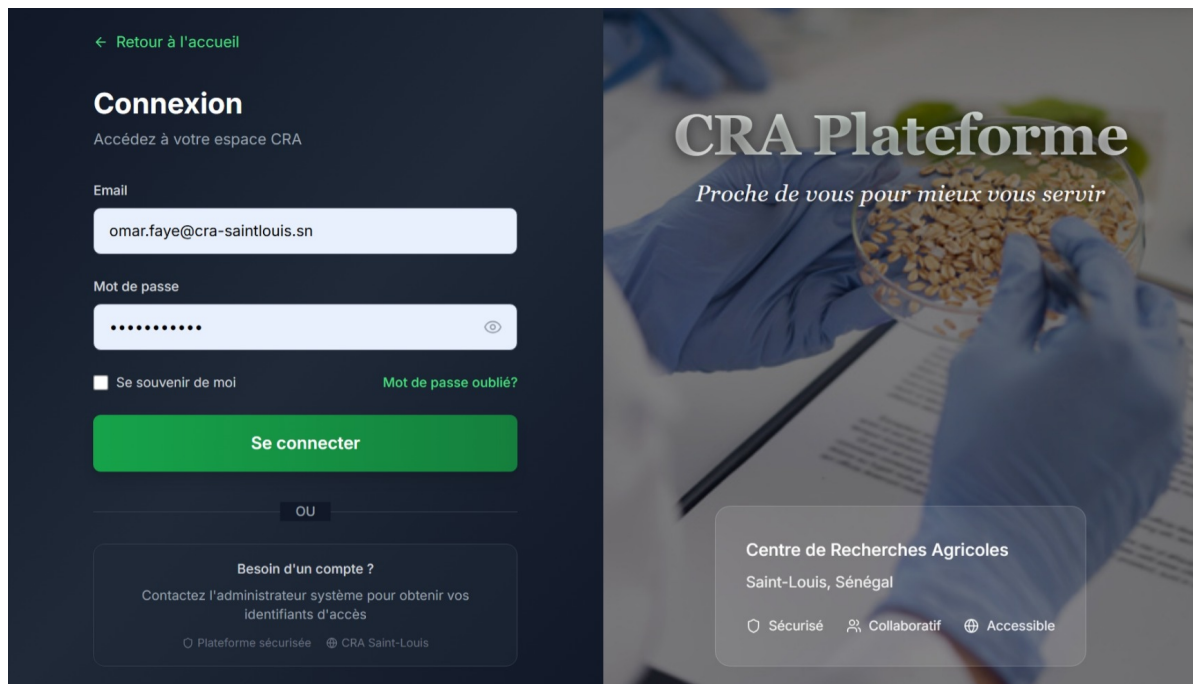


FIGURE 5.6 – Interface d’authentification de la plateforme

5.2.3 Tableau de bord et navigation

Après connexion, l'utilisateur accède à un tableau de bord personnalisé, comme illustré dans la figure 5.7. Ce tableau de bord offre une vue synthétique des informations essentielles, notamment les projets en cours, les documents récents, les activités planifiées et les notifications.

L'interface repose sur une navigation claire et intuitive, facilitant l'accès rapide aux différentes fonctionnalités. Les composants graphiques dynamiques permettent une meilleure lisibilité des données et améliorent l'expérience utilisateur.

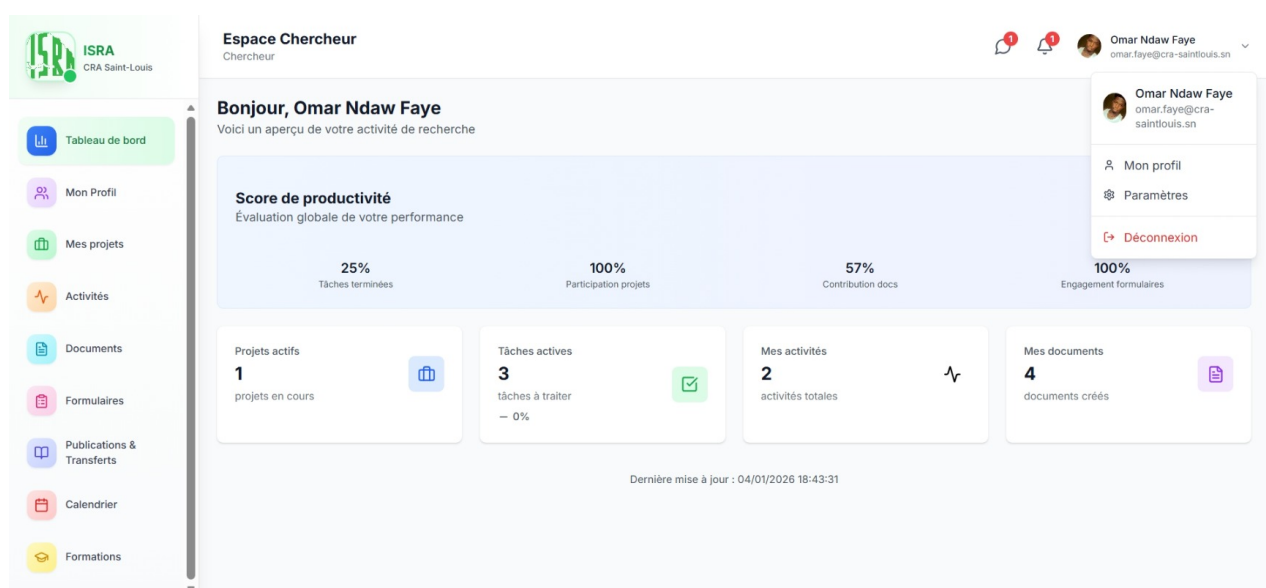


FIGURE 5.7 – Tableau de bord principal de l'utilisateur

5.2.4 Gestion des projets de recherche

La gestion des projets constitue une fonctionnalité centrale de la plateforme. Comme illustré par la figure 5.8, elle permet aux chercheurs de créer, consulter et mettre à jour les projets de recherche menés au sein du CRA.

Chaque projet regroupe des informations structurées telles que les objectifs scientifiques, la durée, les responsables et les activités associées. Cette organisation facilite le suivi des travaux de recherche et améliore la visibilité des projets institutionnels.

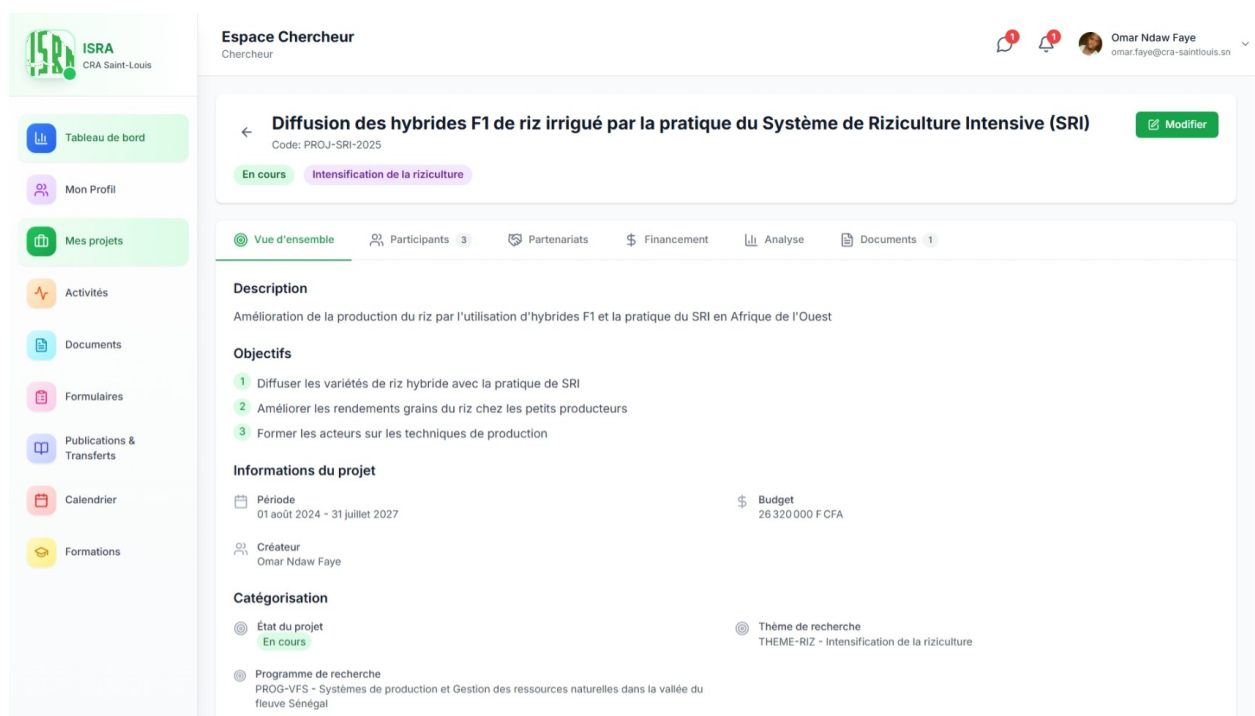


FIGURE 5.8 – Interface de gestion des projets de recherche

5.2.5 Gestion des activités de recherche

La plateforme offre également un module dédié à la gestion des activités de recherche. Ce module permet aux chercheurs d'enregistrer et de suivre les différentes activités liées à un projet, telles que les expérimentations, les collectes de données ou les analyses scientifiques.

Cette fonctionnalité contribue à une meilleure traçabilité des travaux réalisés et facilite la production de rapports d'activités.



FIGURE 5.9 – Gestion des activités de recherche

5.2.6 Rôle du coordonnateur de projet et génération de rapports

Le coordonnateur de projet représente un profil fonctionnel spécifique de la plateforme. Il s'agit d'un chercheur disposant de droits étendus lui permettant d'avoir une vue globale sur l'ensemble des projets et des activités de recherche, au-delà de ses propres travaux.

Contrairement au chercheur standard, le coordonnateur de projet peut consulter l'état d'avancement des projets, suivre les activités réalisées par les différents chercheurs et analyser la progression globale des travaux scientifiques. Cette vision transversale facilite la coordination, la planification et la prise de décision au niveau des projets institutionnels.

Une fonctionnalité supplémentaire essentielle est la **génération de rapports**. Le coordonnateur peut produire des rapports synthétiques à partir des données enregistrées dans le système, notamment :

- rapports d'avancement des projets ;
- bilans d'activités par période ;
- statistiques de production scientifique ;
- rapports destinés à l'administration ou aux partenaires.

Ces rapports sont générés de manière automatisée à partir des données centralisées de la plateforme, garantissant leur fiabilité et leur cohérence. Cette fonctionnalité contribue à améliorer le pilotage des projets de recherche et à renforcer la gouvernance scientifique du CRA.



FIGURE 5.10 – Interface de génération de rapports pour le coordonnateur de projet

5.2.7 Gestion des documents scientifiques

Le module de gestion des documents permet aux chercheurs de déposer, consulter et organiser les documents associés à leurs projets (rapports, données, fichiers scientifiques). Cette fonctionnalité favorise la centralisation des ressources et limite la dispersion des informations.

L'accès aux documents est contrôlé par les rôles utilisateurs, garantissant la confidentialité et la sécurité des données.

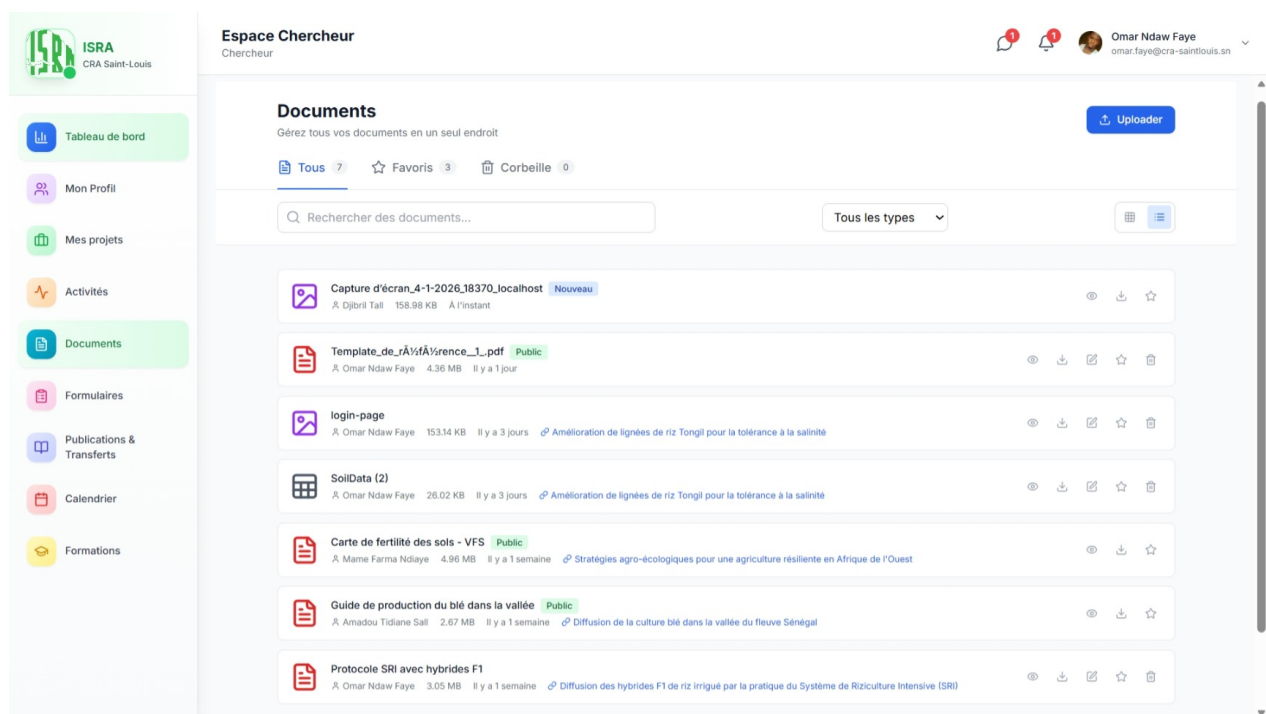


FIGURE 5.11 – Gestion des documents scientifiques pour les chercheurs

5.2.8 Publications et transfert de connaissances

La plateforme intègre un module dédié aux publications scientifiques et au transfert de connaissances. Ce module permet de valoriser les résultats de recherche à travers la publication d'articles, de rapports et de supports de vulgarisation scientifique.

Il constitue un outil stratégique pour renforcer la visibilité des travaux du CRA et faciliter le partage des connaissances avec les partenaires et les acteurs du secteur agricole.

FIGURE 5.12 – Module de publications et de transfert de connaissances

5.2.9 Gestion des formations

La gestion des formations permet de planifier et de suivre les sessions de formation organisées par le CRA. Cette fonctionnalité est destinée aussi bien aux chercheurs qu'aux autres acteurs impliqués dans les activités de renforcement des capacités.

Elle facilite la diffusion des savoirs et contribue à l'amélioration continue des compétences.

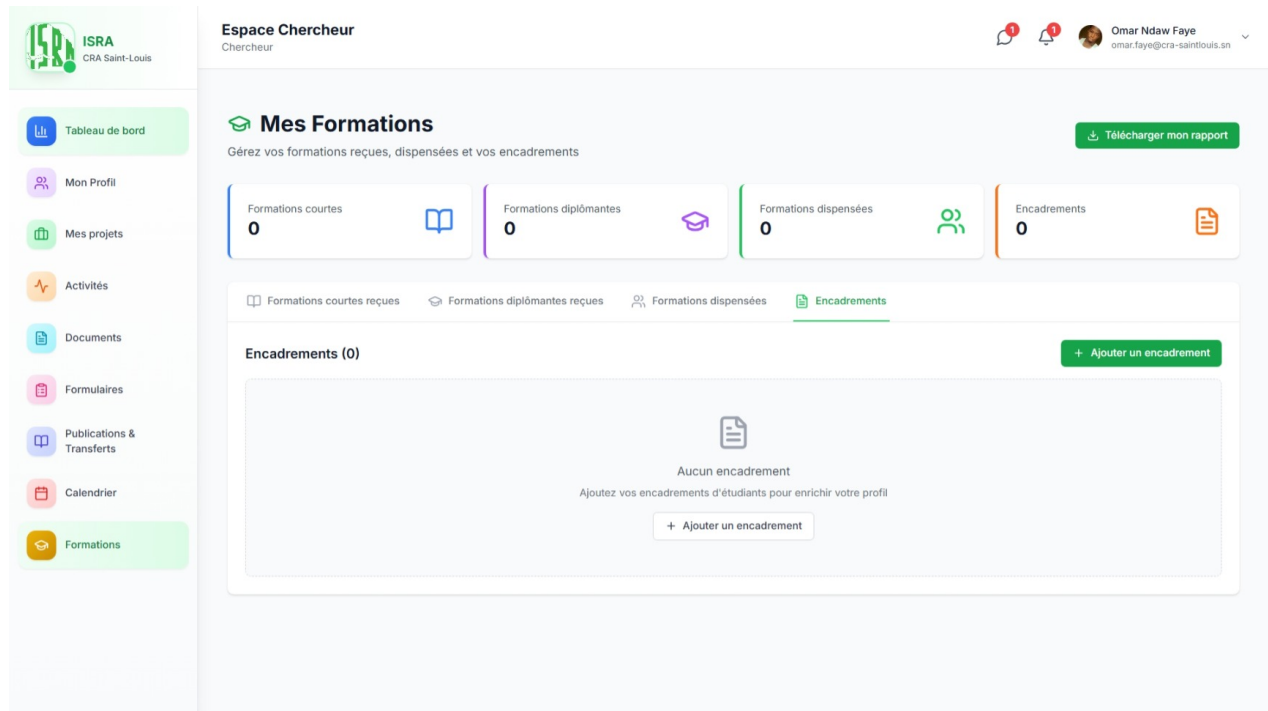


FIGURE 5.13 – Gestion des formations

5.2.10 Système de calendrier

Un système de calendrier intégré permet la planification des événements, réunions, formations et activités de recherche. Ce module améliore la coordination entre les différents acteurs et favorise une meilleure organisation du travail.

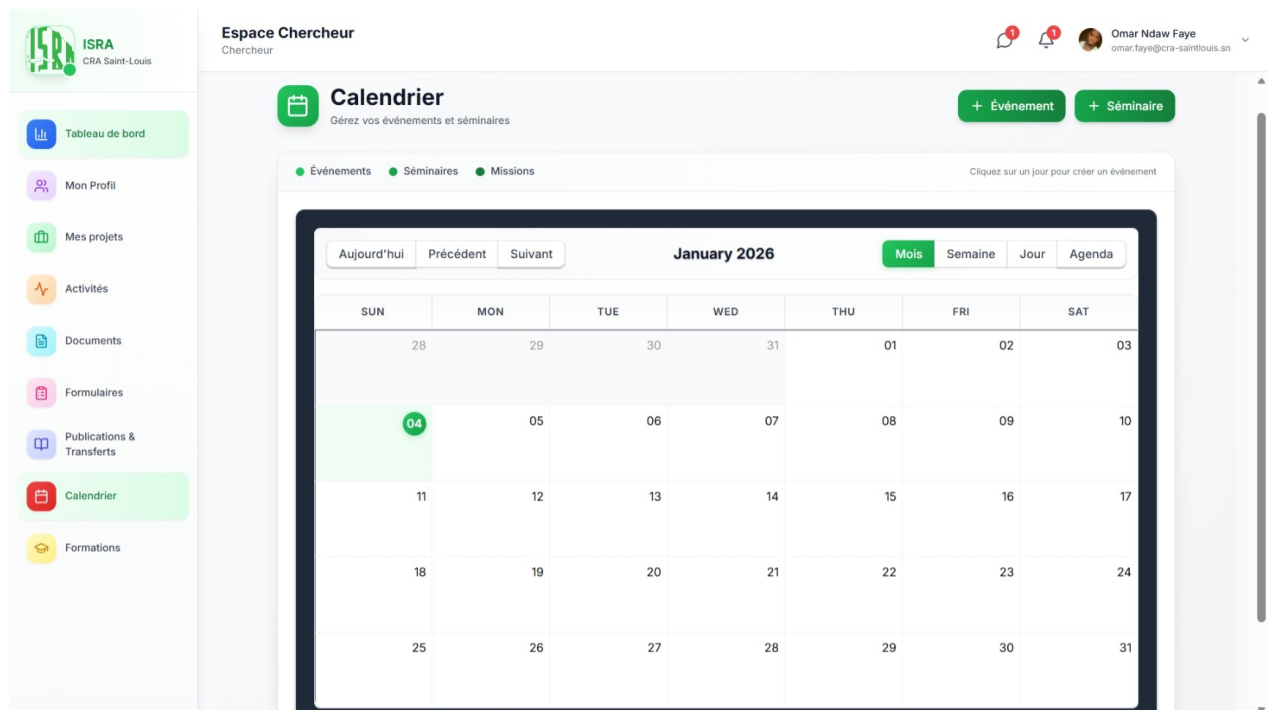


FIGURE 5.14 – Système de calendrier et de planification

5.2.11 Fonctionnalités administratives

Les fonctionnalités administratives sont exclusivement accessibles à l’administrateur du système.

5.2.11.1 Gestion des utilisateurs

Le module de gestion des utilisateurs permet à l’administrateur de créer, modifier et supprimer des comptes utilisateurs, ainsi que d’attribuer les rôles appropriés. Cette gestion centralisée garantit la sécurité et le contrôle des accès.

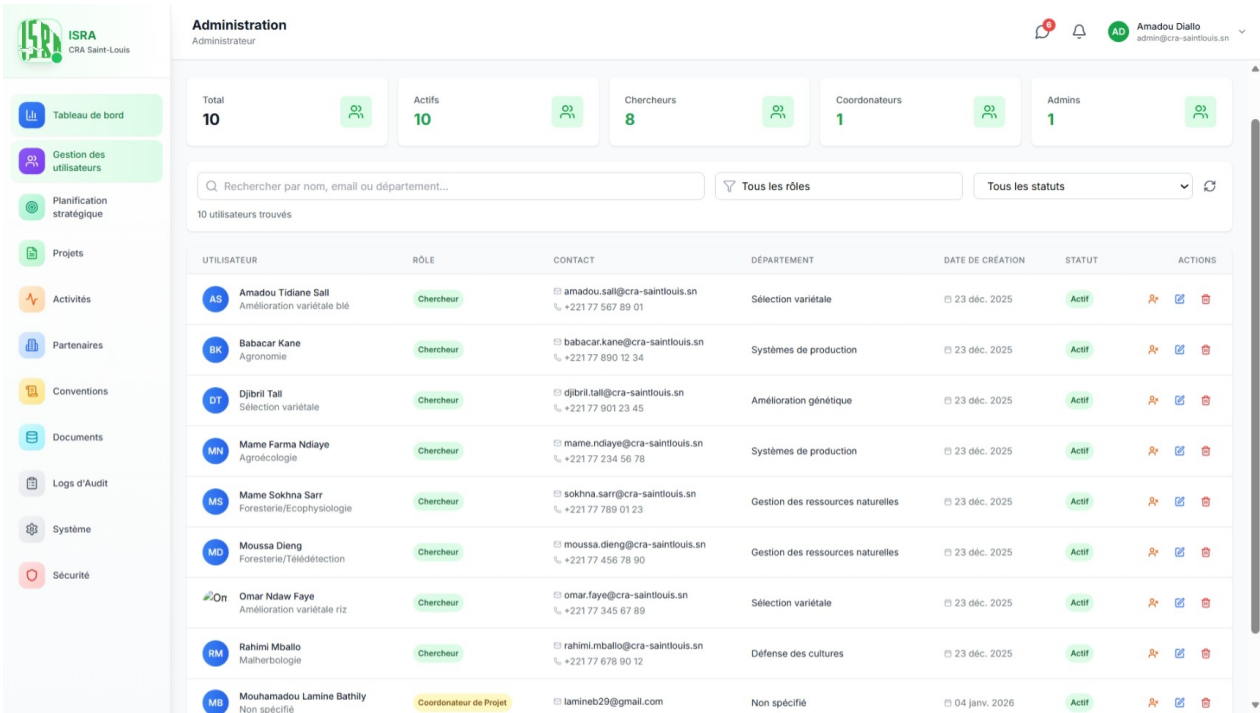


FIGURE 5.15 – Gestion des utilisateurs par l’administrateur

5.2.11.2 Gestion du planning

L’administrateur dispose également d’un module de gestion du planning global, permettant d’avoir une vue d’ensemble sur les activités, formations et événements programmés au sein du CRA.

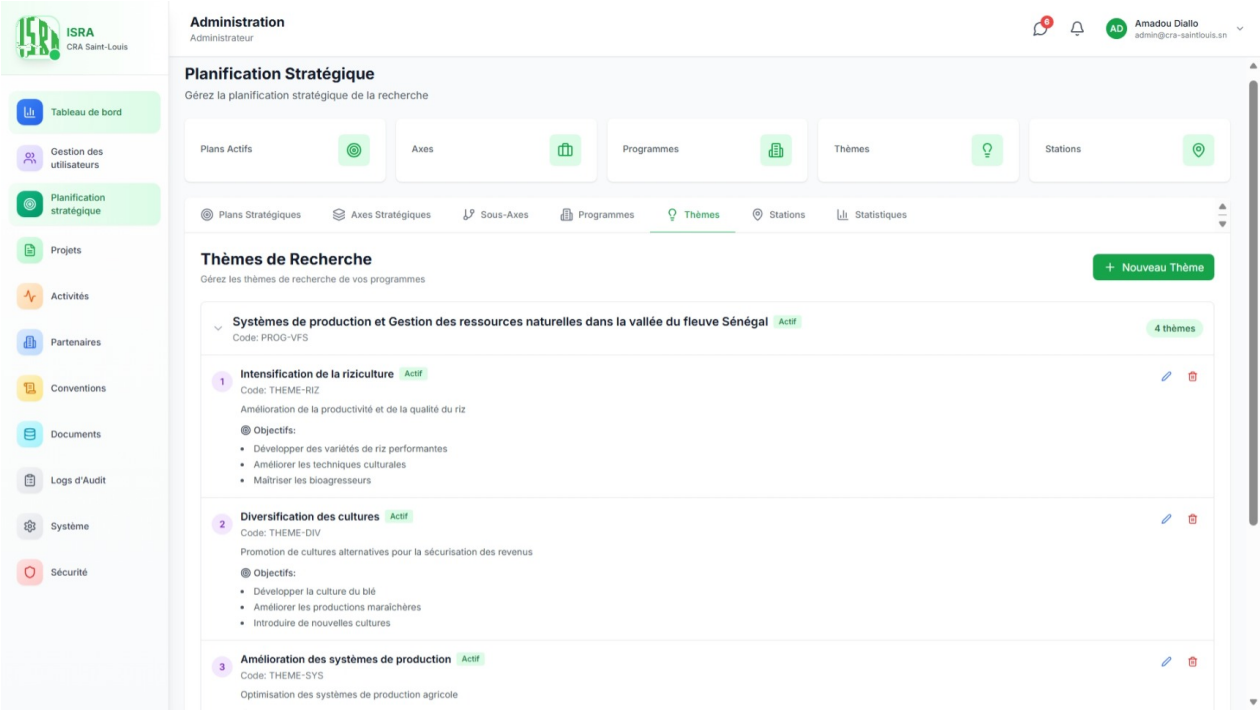


FIGURE 5.16 – Gestion du planning global

5.2.11.3 Gestion des audits

Le module d’audit permet de tracer les actions réalisées sur la plateforme. Il contribue à la transparence, à la sécurité et au contrôle des opérations effectuées par les utilisateurs.

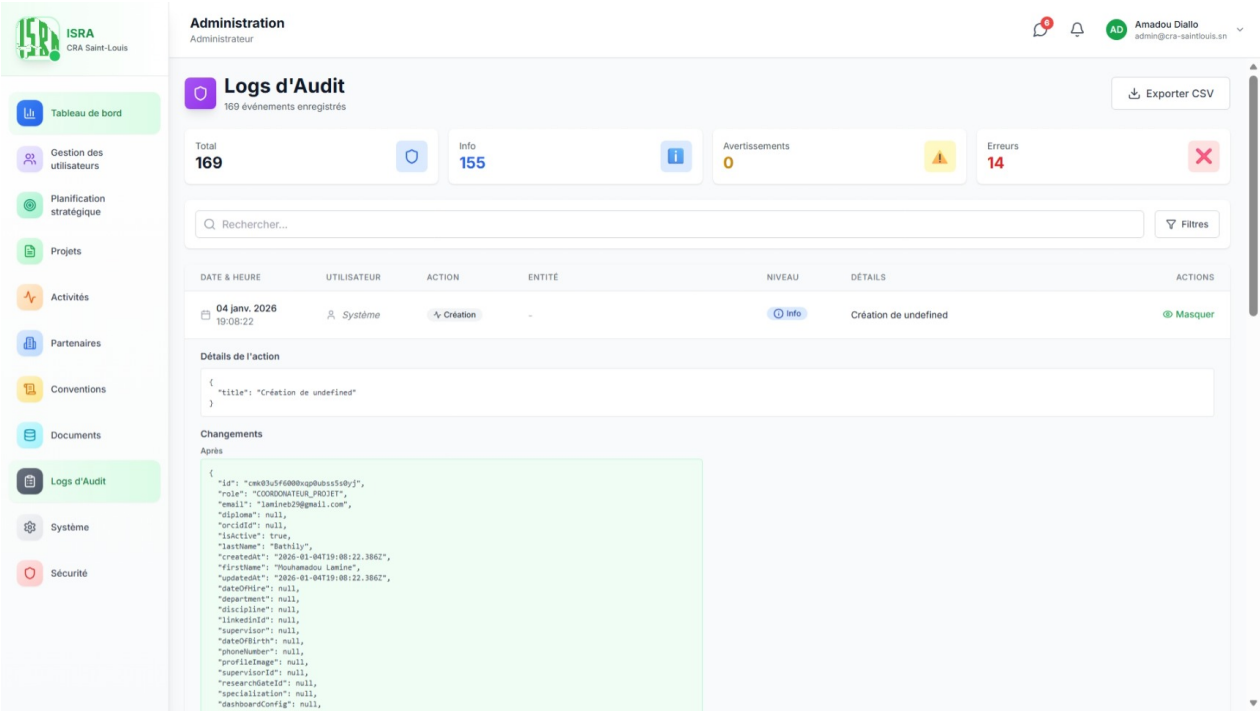


FIGURE 5.17 – Module d’audit des actions utilisateurs

5.2.12 Analyse globale des résultats

L'analyse des résultats obtenus montre que la plateforme répond efficacement aux besoins fonctionnels du CRA. Les fonctionnalités développées permettent une gestion centralisée des projets, une meilleure organisation des activités scientifiques et un contrôle rigoureux des accès.

L'architecture adoptée, combinée à des interfaces ergonomiques, offre une solution robuste, évolutive et adaptée aux exigences institutionnelles du Centre de Recherches Agricoles de Saint-Louis.

Conclusion et Perspectives

Ce mémoire a porté sur la conception d'une plateforme web collaborative destinée à la gestion des activités scientifiques, des données et des documents du Centre de Recherches Agricoles (CRA) de Saint-Louis, structure de l'Institut Sénégalais de Recherches Agricoles (ISRA). Ce travail répond à un besoin réel de digitalisation des processus de recherche afin d'améliorer l'organisation, la centralisation et la valorisation des productions scientifiques.

L'analyse des besoins des différents profils d'utilisateurs a permis de définir des spécifications fonctionnelles et non fonctionnelles adaptées au contexte du CRA. La modélisation UML et la conception architecturale ont contribué à structurer efficacement la solution, tant sur le plan fonctionnel que technique.

La plateforme développée repose sur une architecture 3-tiers, combinant une couche de présentation, une couche métier et une couche d'accès aux données. Le choix de technologies modernes telles que React, Node.js, Express.js, TypeScript, Prisma et PostgreSQL, associé à un modèle de programmation asynchrone, a permis de concevoir une solution performante, évolutive et maintenable. Les résultats obtenus montrent une amélioration notable de la centralisation des informations, de la collaboration entre chercheurs et du suivi des activités scientifiques.

En termes de perspectives, plusieurs évolutions peuvent être envisagées, notamment l'intégration de tableaux de bord décisionnels avancés, l'exploitation de techniques d'analyse et d'intelligence artificielle sur les données collectées, ainsi que l'extension de la plateforme à d'autres centres de recherche agricole. Ces perspectives ouvrent la voie à une solution numérique durable, capable d'accompagner les enjeux actuels et futurs de la recherche agricole.

Références

- (1) 365 DATA SCIENCE MySQL vs PostgreSQL : An Open Source Battle <https://365datascience.com/tutorials/sql-tutorials/mysql-postgresql/> (visité le 13 sept. 2025).
- (2) AGMATIX Field Trial Management Platform - Agronomic Field Data Collection & Analysis <https://www.agmatix.com/ag-field-trial-management/> (visité le 23 août 2025).
- (3) AGMIP The Agricultural Research Data Network (ARDN) <https://agmip.github.io/ARDN/> (visité le 22 août 2025).
- (4) AIRBYTE PostgreSQL vs MySQL : A Detailed Comparison for Data Engineers <https://airbyte.com/data-engineering-resources/postgresql-vs-mysql> (visité le 23 sept. 2025).
- (5) ANARA Best Research Collaboration Software & Tools <https://anara.com/blog/research-collaboration-tools> (visité le 13 sept. 2025).
- (6) APPY PIE PostgreSQL vs MySQL - Which One Reigns Supreme ? <https://www.appypieautomate.ai/blog/postgresql-vs-mysql> (visité le 23 sept. 2025).
- (7) AUTHGEAR JWT Authentication : A Comprehensive Guide for Developers <https://www.authgear.com/post/jwt-authentication-a-secure-scalable-solution-for-modern-applications> (visité le 23 sept. 2025).
- (8) AWS Building a Three-Tier Architecture on a Budget <https://aws.amazon.com/blogs/architecture/building-a-three-tier-architecture-on-a-budget/> (visité le 13 sept. 2025).
- (9) BROWSERSTACK REST API Security : Design Principles and Best Practices <https://www.browserstack.com/guide/rest-api-design-principles-and-best-practices> (visité le 13 sept. 2025).
- (10) CGIAR CGIAR Platform for Big Data in Agriculture <https://bigdata.cgiar.org> (visité le 23 août 2025).
- (11) CGIAR Big Data Platform and One CGIAR <https://bigdata.cgiar.org/big-data-platform-and-one-cgiar/> (visité le 23 août 2025).

- (12) CGIAR PLATFORM FOR BIG DATA IN AGRICULTURE GARDIAN - Global Agricultural Research Data Innovation & Acceleration Network <https://bigdata.cgiar.org/resources/gardian/> (visité le 22 août 2025).
- (13) DAILY.DEV RESTful API Design Best Practices Guide 2024 <https://daily.dev/blog/restful-api-design-best-practices-guide-2024> (visité le 13 sept. 2025).
- (14) DELVE What's the Best QDA Software for Collaborative Qualitative Research <https://delvetool.com/blog/best-qda-software-collaborative-qualitative-research> (visité le 13 sept. 2025).
- (15) FAO AGRIS - International System for Agricultural Science and Technology <https://agris.fao.org> (visité le 23 août 2025).
- (16) GEEKSFORGEEKS MVC Design Pattern <https://www.geeksforgeeks.org/system-design/mvc-design-pattern/> (visité le 13 sept. 2025).
- (17) GEEKSFORGEEKS MVC Framework Introduction <https://www.geeksforgeeks.org/software-engineering/mvc-framework-introduction/> (visité le 23 sept. 2025).
- (18) GEEKSFORGEEKS Comparison of Popular JavaScript ORMs : Prisma, TypeORM, and Sequelize <https://www.geeksforgeeks.org/javascript/comparison-of-popular-javascript-orms-prisma-typeorm-and-sequelize/> (visité le 13 sept. 2025).
- (19) HEVO DATA REST API Best Practices <https://hevodata.com/learn/rest-api-best-practices/> (visité le 22 sept. 2025).
- (20) HEYMARVIN Research Collaboration Software : What to Know Before Choosing <https://heymarvin.com/resources/research-collaboration-software/> (visité le 13 sept. 2025).
- (21) IFPRI CGIAR Platform for Big Data in Agriculture <https://rdm.ifpri.info/cgiar-platform-for-big-data-in-agriculture/> (visité le 23 août 2025).
- (22) LEAPCELL Navigating Node.js ORMs : Prisma, TypeORM, Sequelize <https://leapcell.io/blog/navigating-node-js-orms-prisma-typeorm-sequelize> (visité le 23 sept. 2025).
- (23) LIQUIBASE PostgreSQL vs MySQL 2025 : Complete Database Comparison <https://www.liquibase.com/resources/guides/postgresql-vs-mysql> (visité le 13 sept. 2025).
- (24) META React - The Library for Web and Native User Interfaces <https://react.dev> (visité le 23 sept. 2025).
- (25) PRISMA Prisma ORM vs Sequelize <https://www.prisma.io/docs/orm/more/comparisons/prisma-and-sequelize> (visité le 23 sept. 2025).
- (26) RESEARCHMATE Top 6 Best Research Collaboration Tools <https://researchmate.net/top-6-best-research-collaboration-tools/> (visité le 13 août 2025).

- (27) RESTFULAPI.NET REST API Best Practices <https://restfulapi.net/rest-api-best-practices/> (visité le 13 sept. 2025).
- (28) ROYER, R. (2024). How Data Management Platforms Are Driving Digital Agronomy. iGrow News.
- (29) SIMPLEBACKUPS MySQL vs PostgreSQL : A Comprehensive Comparison <https://simplebackups.com/blog/mysql-vs-postgresql-a-comprehensive-comparison-of-the-most-popular-databases> (visité le 23 sept. 2025).
- (30) SOFTWARE SECURED How to Properly Secure Your JWTs <https://www.softwaresecured.com/post/how-to-properly-secure-your-jwts> (visité le 23 sept. 2025).
- (31) STRAPI RESTful API Design Guide : Principles and Best Practices <https://strapi.io/blog/restful-api-design-guide-principles-best-practices> (visité le 23 sept. 2025).
- (32) SUPERTOKENS What is JWT? Understand JSON Web Tokens <https://supertokens.com/blog/what-is-jwt> (visité le 23 sept. 2025).
- (33) TALENT500 What Is MVC Architecture? Model-View-Controller Explained <https://talent500.com/blog/mvc-architecture-beginner-expert-guide/> (visité le 13 sept. 2025).
- (34) TECHTARGET What is a 3-Tier Application Architecture? <https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application> (visité le 23 sept. 2025).
- (35) VAADATA JWT : Vulnerabilities, Attacks & Security Best Practices <https://www.vaadata.com/blog/jwt-json-web-token-vulnerabilities-common-attacks-and-security-best-practices/> (visité le 23 sept. 2025).
- (36) VFUNCTION Understanding the Architecture of a 3-tier Application <https://vfunction.com/blog/3-tier-application/> (visité le 13 sept. 2025).
- (37) WIKIPEDIA Multitier Architecture https://en.wikipedia.org/wiki/Multitier_architecture (visité le 13 sept. 2025).
- (38) WU, P.-F. et al. (2011). An Adaptable XML Based Approach for Scientific Data Management and Integration. PMC 3098743.